# 第 1 章　绪论

**一、单项选择题**

1．①B　②D。
2．C。
3．A。
4．B
5．　C
6．A。
7．C A
8．C。
9．　C
10．C

**二、判断题（在各题后填写“√”或“×”）**

1. 线性结构只能用顺序结构来存放，非线性结构只能用非顺序结构来存放。（×）
2. 数据元素是数据的最小单位。(×　　)
3. 记录是数据处理的最小单位。（　×　）
4. 算法就是程序。（×）
5. 数据的逻辑结构是指数据的各数据项之间的逻辑关系；(×　)
6. 数据的物理结构是指数据在计算机内的实际存储形式。(√　)
7. 在顺序存储结构中，有时也存储数据结构中元素之间的关系。(×　　)
8. 顺序存储方式的优点是存储密度大，且插入、删除运算效率高。(　×　)
9. 线性表若采用链式存储结构时，要求内存中可用存储单元的地址一定是不连续的。(×　　)
10. 数据的逻辑结构说明数据元素之间的顺序关系,它依赖于计算机的储存结构.（　×　）

**三、填空题**

1. 逻辑结构　　物理结构　　操作（运算）　　算法
2. 集合　线性结构　　树形结构　　图状结构或网状结构
2. 没有　　1　　没有　　1
4. 前驱　　1　　后续　　任意多个
5. 表示（又称映像）
6.顺序存储方式　　链式存储方式
7.（1）逻辑特性　（2）在计算机内部如何表示和实现　（3）数学特性
8. 算法的时间复杂度和空间复杂度
9.（1）有穷性　（2）确定性　（3）可行性
10. 1　$\log_2 n$　n　$n^2$　$2^n$　实际不可计算　　高效
11. (n+3)(n-2)/2
11. ① (1)1　(2)1　(3)f(m,n-1)　(4)n　　② 9
13. o($\log_2 n$)。

**四、应用题**

1．解答：（1）图略。线性结构

（2）图略。树结构
（3）图略。图结构

2. 将学号、姓名、平均成绩看成一个记录（元素，含三个数据项），将 100 个这样的记录存于数组中。因一般无增删操作，故宜采用顺序存储。

```
typedef  struct
  {int num;//学号
   char name[8];//姓名
   float score;/平均成绩
  }node;
 node student[100];
```

3. 第一层 FOR 循环判断 n+1 次，往下执行 n 次，第二层 FOR 执行次数为(n+(n-1)+(n-2)+…+1)，第三层循环体受第一层循环和第二层循环的控制，其执行次数如下表：

| i= | 1 | 2 | 3 | … | n |
|---|---|---|---|---|---|
| j=n | n | n | n | … | n |
| j=n-1 | n-1 | n-1 | n-1 | … | |
| … | … | … | … | | |
| j=3 | 3 | 3 | | | |
| j=2 | 2 | 2 | | | |
| j=1 | 1 | | | | |

执行次数为(1+2+…+n)+(2+3+…+n)+…+n=n*n(n+1)/2-n($n^2$-1)/6。在 n=5 时，f(5)=55，执行过程中，输出结果为：sum=15,sum=29,sum=41,sum=50,sum=55（每个 sum= 占一行,为节省篇幅，这里省去换行）。

4. 解答：（1）int locate(dataytpe A[1..n],dateytpe k)

```
{  i=1;
   while ((i<=n)&&(A[i]!=k)) i++;
   if (i<=n)    return(i);
   else        return(o);
 }
```

最坏时间复杂度 T（n）=O(n).

（2）Void CZ_max(datatype A[n],x,y)

```
{ x=A[1]; y=A[1];
 for(i=2;i<=n;I++)
   if(x<A[i]{y=x;x=A[i];}            /*替换最大值*/
   else if(y<A[i] y=A[i];             /*替换次最大值*/
}
```

最坏情况时间复杂度 T(n)=O(n).

# 第 2 章　线性表

**一、单项选择题**

1．B
2.A
3．C
4．C
5．C
6．B
7．B
8．.A
9．D
10. B
11. B
12．B
13.C

**二、判断题（在各题后填写“√”或“×”）**

1. 链表中的头结点仅起到标识的作用。(×　)
2. 线性表采用链表存储时，结点和结点内部的存储空间可以是不连续的。(　√　)
3. 顺序存储方式插入和删除时效率太低，因此它不如链式存储方式好。(　×　)
4. 对任何数据结构链式存储结构一定优于顺序存储结构。(× )
5. 所谓静态链表就是一直不发生变化的链表。(　×　)
6. 线性表的特点是每个元素都有一个前驱和一个后继。(　×　)
7. 循环链表不是线性表. (　×　)
8. 线性表只能用顺序存储结构实现。(　×　)
9. 线性表就是顺序存储的表。(　×　)
10. 链表是采用链式存储结构的线性表,进行插入、删除操作时，在链表中比在顺序存储结构中效率高。(√　)

**三、填空题**

1．<u>　　必定　　</u>　　<u>　　不一定　　</u>
2．<u>　　头指针　　</u>　　<u>　　头结点指针域　　</u>　　<u>　　前驱结点指针域　　</u>
3．　双向链表
4．　n　　O(n)　　n/2　　O(n)
5．. 单链表　　循环链表　　双向链表
6．　指针
7．(n-1) /2
8．py->next=px->next; px->next=py
9．4　　2
10．　i=1;　　i≤L.last
11．(1)L->next=null　　//置空链表,然后将原链表结点逐个插入到有序表中
　(2)p!=null　　//当链表尚未到尾，p 为工作指针
　(3)q!=null　　//查 p 结点在链表中的插入位置，这时 q 是工作指针。
　(4)p->next=r->next　　//将 p 结点链入链表中

(5)r->next=p　　//r 是 q 的前驱，u 是下个待插入结点的指针。

**四、应用题**

1. 在线性表的链式存储结构中，头指针指链表的指针，若链表有头结点则是链表的头结点的指针，头指针具有标识作用，故常用头指针冠以链表的名字。头结点是为了操作的统一、方便而设立的，放在第一元素结点之前，其数据域一般无意义（当然有些情况下也可存放链表的长度、用做监视哨等等），有头结点后，对在第一元素结点前插入结点和删除第一结点，其操作与对其它结点的操作统一了。而且无论链表是否为空，头指针均不为空。首元结点也就是第一元素结点，它是头结点后边的第一个结点。

2. 在单链表中不能从当前结点（若当前结点不是第一结点）出发访问到任何一个结点，链表只能从头指针开始，访问到链表中每个结点。在双链表中求前驱和后继都容易，从当前结点向前到第一结点，向后到最后结点，可以访问到任何一个结点。

3.

1）本算法功能是将双向循环链表结点的数据域按值自小到大排序，成为非递减（可能包括数据域值相等的结点）有序双向循环链表。

2）(1)r->prior=q->prior;//将 q 结点摘下，以便插入到适当位置。

(2)p->next->prior=q;//（2）（3）将 q 结点插入

(3)p->next=q;

(4)r=r->next;或 r=q->next;//后移指针，再将新结点插入到适当位置。

**五、算法设计题**

1.［题目分析］在递增有序的顺序表中插入一个元素 x，首先应查找待插入元素的位置。因顺序表元素递增有序，采用折半查找法比顺序查找效率要高。查到插入位置后，从此位置直到线性表尾依次向后移动一个元素位置，之后将元素 x 插入即可。

```
void  Insert（ElemType A[]，int size, ElemType x）
 //  A 是有 size 个元素空间目前仅有 num（num<size）个元素的线性表。本算法将元素 x 插入到线性表中，并保持线性表的有序性。
    {low=1；high=num；  //题目要求下标从 1 开始
     while（low<=high）//对分查找元素 x 的插入位置。
       {mid=（low+high）/2；
        if（A[mid]==x）{low=mid+1；break；}
        else if（A[mid]>x）high=mid-1 ；else  low=mid+1 ；
        }
      for（i=num；i>=low；i--）  A[i+1]=A[i]；//元素后移。
      A[i+1]=x；  //将元素 x 插入。
    }算法结束。
```

［算法讨论］算法中当查找失败（即线性表中无元素 x）时，变量 low 在变量 high 的右面（low=high+1）。移动元素从 low 开始，直到 num 为止。特别注意不能写成 for（i=low；i<=num；i++）A[i+1]=A[i]，这是一些学生容易犯的错误。另外，题中未说明若表中已有值为 x 的元素时不再插入，故安排在 A[mid]= =x 时，用 low（=mid+1）记住位置，以便后面统一处理。查找算法时间复杂度为 O（logn），而插入时的移动操作时间复杂度为 O（n），若用顺序查找，则查找的时间复杂度亦为 O（n）。

2．解答：(1)顺序表

分析：将顺序表的第一个元素与最后一个元素互换，第二个元素与倒数第二个元素互换。

```
void   invert(SeqList   *L,   int   *num)
       {
          int   j;
          ElemType   tmp;
         for(j=0;j<=(*num-1)/2;j++)
         { tmp=L[j];
           L[j]=L[*num-j-1];
           L[*num-j-1]=tmp;}
      }
```

(2)单链表

```
void   invert(LinkList   L)
   {
    Node   *p, *q, *r;
       if(L->next ==NULL)   return;                    /*链表为空*/
       p=L->next;
       q=p->next;
    p->next=NULL;                              /* 摘下第一个结点,生成初始逆置表 */
    while(q!=NULL)                             /* 从第二个结点起依次头插入当前逆置
表 */
     {
      r=q->next;
      q->next=L->next;
      L->next=q;
      q=r;
   }
 }
```

3．解答：(1)定位 LOCATE(L，X)

在带头结点类单链表上实现的算法为：

```
int locate_lklist(lklist head,datatype x)
/*求表 head 中第一个值等于 x 的的序号，不存在这种结点时结果为 0*/
{p=head->next;j=1;          /*置初值*/
while((p!=NULL)&&(p->data!=x))
{p=p->next;j++}/*未达表结点又未找到值等于 X 的结点时经，继续扫描*/
if (p->data = =x)   return(j);
else           return(0);
}
```

在无头结点的单链表上实现的算法为：

```
int Wlocate(lklist head,datatype X)
/*求表 head 中第一个值等于 x 的结点的序号。不存在这种结点时结果为 0*/
{p=head; j=1;                    /*置初值*/
while((p!=NULL)&&(p->data!=x))
{p=p->next;j++}/*未达表结点又未找到值等于 X 的结点时经，继续扫描*/
if( p->data = =X)   return(j);
```

```
    else            return(0);
    }
```

(2)按序号查找 find(L,i)

在带头结点的单链表上实现的算法为：

```
pointer  find_lklist(lklist head , int i);
{ j=1; p=head->next;
while((j<1)&&(p!=NULL)){p=p->next; j++}
if(i= = j) return(p);
 else   return(NULL);
}
```

在无头结点的单链表上实现的算法为：

```
pointer  find_lklist(lklist head , int i);
{ j=1; p=head;
while((j<1)&&(p!=NULL)){p=p->next; j++}
if(i= = j) return(p);
 else   return(NULL);
}
```

(3)、插入 INSERT(L，X，i)

在带头结点单链表上实现的算法为：

```
  void insert_lklist(lklist head,datatype x,int I)
/*在表 haed 的第 i 个位置上插入一人以 x 为值的新结点*/
{p=find_lklist(head,i-1);  /*先找第 i-1 个结点*/
if(p= =NULL)reeor(“不存在第 i 个位置”)/*若第 i-1 个结点不存在，退出*/
else{s=malloc(size);s->data=x      /*否则生成新结点*/
    s->next=p->next   /*结点*p 在链域值传给结点*s 的链域*/
    p->next=s;       /*修改*p 的链域*/
    }
    }
```

在无头结点的单链表上实现的算法为：

```
void Winsert(lklist head,dataytpe X,int i)
/*在表 haed 的第 i 个位置上插入一人以 x 为值的新结点*/
{if(i<=0) error(“i<=0”);
  else{ s=malloc(size);s->data=X;  /*否则生成新结点*/
  if(i= =1){s->next=head;head=s;}
   else{ p=wfind_lklist(lklist head,i-1);
     if(p= =NULL) error(“i>n+1”);
       else{s->next=p->next;p->next=s;}
     }
}
```

(4)删除 DELDTE(L,i)

在带头结点的单链表上实现的算法为：

```
void delete_lklist(lklist head,int i) /*删除表 head 的第 i 个结点*/
{p=find_lklist(head,i-1)   /*先找待删结点的直接前驱*/
if((p!==NULL)&&(p->next!=NULL))/*若直接前趋存在且待结点存在*/
```

```
(q=p->next; /*q 指向待删结点*/
p->next=q->next/*摘除待结点*/;
free(q);/*释放已摘除结点 q*/
}
else error(“不存在第 i 个结点”)/*否则给出相关信息*/
}
```

在无头结点的单链表上实现的算法为：

```
void Wdelete(lklist head,int i)
/*  删除表 head 的第 i 个结点，若该链表仅有一个结点时，赋该结点指针 NULL*/
{if(i<=0) error(“I＜＝0”
else{if(i= =0){q=head;head=head->next;free(q);}
    else{p=wfind_lklist(head,i-1);/*找链表 head 中第 i-1 结点指针*/
    if(p!=NULL)&&(p->next!=NULL)
     {q=p->next; p->next=q->next; free(q);}
    else error(“不存在第 I 个结点”)；
    }
    }
}
```

4.【解答】算法如下：

```
LinkList   merge(LinkList   A,   LinkList B,   LinkList   C)
{ Node   *pa, *qa, *pb, *qb, *p;
   pa=A->next;                                    /*pa 表示 A 的当前结点*/
   pb=B->next;
  p=A;   / *利用 p 来指向新连接的表的表尾，初始值指向表 A 的头结点*/

   while(pa!=NULL   &&   pb!=NULL)     /*利用尾插法建立连接之后的链表*/
  {     qa=pa->next;
      qb=qb->next;
       p->next=pa;     /*交替选择表 A 和表 B 中的结点连接到新链表中；*/
      p=pa;
      p->next=pb;
      p=pb;
      pa=qa;
      pb=qb;
  }
  if(pa!=NULL)     p->next=pa;           /*A 的长度大于 B 的长度*/
  if(pb!=NULL)     p->next=pb;           /*B 的长度大于 A 的长度*/
  C=A;
  Return(C);
}
```

5.［题目分析］在顺序存储的线性表上删除元素，通常要涉及到一系列元素的移动（删第 i 个元素，第 i+1 至第 n 个元素要依次前移）。本题要求删除线性表中所有值为 item 的数据元素，并未要求元素间的相对位置不变。因此可以考虑设头尾两个指针（i=1，j=n），从两端向中间移动，凡遇到值 item 的数据元素时，直接将右端元素左移至值为 item 的数据元素位

置。

```
void  Delete（ElemType A[ ]，int  n）
//A 是有 n 个元素的一维数组，本算法删除 A 中所有值为 item 的元素。
{i=1；j=n；//设置数组低、高端指针（下标）。
 while（i<j）
   {while（i<j && A[i]!=item）i++;          //若值不为 item，左移指针。
    if（i<j）while（i<j && A[j]==item）j--；//若右端元素值为 item，指针左移
    if（i<j）A[i++]=A[j--];
   }
```

[算法讨论] 因元素只扫描一趟，算法时间复杂度为 O（n）。删除元素未使用其它辅助空间，最后线性表中的元素个数是 j。若题目要求元素间相对顺序不变，请参见本章三、填空题 25 的算法。

6. 分析设置两个指针，分别指向*S 及其后继，然后按循环链表特性，顺序往下查找*s 的直接前趋，找到后删除；

```
void DELETE_Xlklist(lklist S)
{p=s; q=p->next;
while (q->nest!=s)
  { p=q; q=q->next;}
p->next=s; free(q);
}
```

7. 分析：在链表 L 中依次取元素，若取出的元素是字母，把它插入到字母 B 中，然后在 L 中删除该元素；若取出的元素是数字，把它插入到数字链 D 中，然后在 L 中删除该元素。继续取下一个元素，直到链表的尾部。最后 B、D、L 中分别存放的是字母字符、数字字符和其它字符。

设原表有头结点、头指针 L，新建数字字符链 D，字母字符链 B，其它字符链 R。

```
void DISM_lklist(lklist L,lklist D,lklist B,lklist R)
{ D =malloc(size of(int)); D ->next= D; /*建 D 循环链表头结点*/
 B =malloc(sizeof(char)); B ->next= B; /*建 B 循环链表头结点*/
p= L;q=p->next;
while(q!=null)
 {if((q->data<=' 9' )&&(q->data>=' 0' ))
  {p->next=q->next; /*在表 L 中摘除 q 结点*/
   q->next= D ->next; D ->next=q; /*将 q 结点插入 D 中*/
   q=p->next;   /*移动 q 指针*/
  }
else if (q->data<=' z' )&&(q->data>=' a' )||(q->data<=' z' )&&(q->data>=' a' ))
   {p->next=q->next; /*在表 L 中删除 q 结点*/
    q->next= B ->next; B ->next=q; /*将 q 结点插入 B 中*/
    q=p->next;    /*移动 q 指针*/
    }else {p=q;q=p->next;}         /*移动 q 指针*/
    }
    p->next=L;R=L;            /*使 R 为循环表*/
```

```
}
```

8. 约瑟夫环问题

【解答】算法如下：

```
typedef struct Node
{
  int password;
  int num;
  struct Node *next;
}   Node,*Linklist;

void Josephus()
{
   Linklist L;
   Node *p,*r,*q;
   int m,n,C,j;
   L=(Node*)malloc(sizeof(Node));   /*初始化单向循环链表*/
   if(L==NULL) { printf("\n 链表申请不到空间!");return;}
   L->next=NULL;
   r=L;
   printf("请输入数据 n 的值(n>0):");
   scanf("%d",&n);
   for(j=1;j<=n;j++)
/*建立链表*/
     {
                p=(Node*)malloc(sizeof(Node));
           if(p!=NULL)
                {
                              printf("请输入第%d 个人的密码:",j);
                   scanf("%d",&C);
                   p->password=C;
                   p->num=j;
                   r->next=p;
                 r=p;
         }
     }
   r->next=L->next;
  printf("请输入第一个报数上限值 m(m>0):");
   scanf("%d",&m);
   printf("*****************************************\n");
   printf("出列的顺序为:\n");
   q=L;
   p=L->next;
   while(n!=1)                                          /*计算出列的顺序*/
   {
                 j=1;
             while(j<m)                                 /*计算当前出列的
人选 p*/
             {
                                q=p;                    /*q 为当前结点
p 的前驱结点*/
```

```
                        p=p->next;
                        j++;
            }
            printf("%d->",p->num);
            m=p->password;                                  /*获得新密码*/
            n--;
            q->next=p->next;        /*p 出列*/
            r=p;
            p=p->next;
            free(r);
        }
    printf("%d\n",p->num);
}
```

# 第 3 章　栈和队列

## 一　单项选择题

1. B A B
2. D
3. D
4. C
5. A
6. C
7. B
8. B
9. B
10. D
11. C
12. C

## 二、填空题

1. 操作受限（或限定仅在表尾进行插入和删除操作）　　后进先出
2. 3 1 2
3. S×SS×S××
4. 假溢出时大量移动数据元素
5. 先进先出
6. s=(LinkedList)malloc(sizeof(LNode))；　　s->data=x;　　s->next=r->next；
   r->next=s;　　r=s;
7. 栈
8. (rear-front+m) % m;
9. ls=NULL
10. p->data=x,　　ls=p
11. p->data,　　free(p)
12. p->data,　　p,　　lq->rear=p

## 三、应用题

1.【解答】(1) 顺序栈　(top 用来存放栈顶元素的下标)

判断栈 S 空：如果 S->top==-1 表示栈空。

判断栈 S 满：如果 S->top==Stack_Size-1 表示栈满。

(2) 链栈（top 为栈顶指针，指向当前栈顶元素前面的头结点）

判断栈空：如果 top->next==NULL 表示栈空。

判断栈满：当系统没有可用空间时，申请不到空间存放要进栈的元素，此时栈满。

2. 设操作数栈是 opnd，操作符栈是 optr，对算术表达式 A-B*C/D-E↑F 求值，过程如下：

| 步骤 | opnd 栈 | optr 栈 | 输入字符 | 主要操作 |
|---|---|---|---|---|
| 初始 | | # | A-B*C/D-E↑F# | PUSH(OPTR,’#’) |
| 1 | A | # | A-B*C/D-E↑F# | PUSH(OPND,A) |
| 2 | A | # - | -B*C/D-E↑F# | PUSH(OPTR,’-’) |
| 3 | AB | # - | B*C/D-E↑F# | PUSH(OPND,B) |

| | | | | |
|---|---|---|---|---|
| 4 | AB | # - * | *C/D-E↑F# | PUSH(OPTR,’*’) |
| 5 | ABC | # - * | C/D-E↑F# | PUSH(OPND,C) |
| 6 | AT(T=B*C) | # - / | /D-E↑F# | PUSH(OPND,POP(OPND)*POP(OPND))<br>PUSH(OPTR,’/’) |
| 7 | ATD | # - / | D-E↑F# | PUSH(OPND,D) |
| 8 | AT(T=T/D)<br>T(T=A-T) | # -<br># - | -E↑F# | x=POP(OPND);y=POP(OPND)<br>PUSH(OPND,y/x);<br>x=POP(OPND);y=POP(OPND);<br>PUSH(OPND,y-x)<br>PUSH(OPTR,’-’) |
| 9 | TE | # - | E↑F# | PUSH(OPND,E) |
| 10 | TE | # -↑ | ↑F# | PUSH(OPTR, ‘↑’) |
| 11 | TEF | # -↑ | F# | PUSH(OPND,F) |
| 12 | TE<br>TS(S=E↑F)<br><br>R(R=T-S) | <br>#-<br><br># | # | X=POP(OPND) Y=POP(OPND)<br>POP(OPTR) PUSH(OPND,y↑x)<br>x=POP(OPND) y=POP(OPND)<br>POP(OPTR) PUSH(OPND,y-x) |

3. 设栈 S1 和栈 S2 共享向量 V[1..m]，初始时，栈 S1 的栈顶指针 top[0]=0，栈 S2 的栈顶指针 top[1]=m+1,当 top[0]=0 为左栈空,top[1]=m+1 为右栈空;当 top[0]=0 并且 top[1]=m+1 时为全栈空。当 top[1]-top[0]=1 时为栈满。

4. 设顺序存储队列用一维数组 q[m]表示，其中 m 为队列中元素个数，队列中元素在向量中的下标从 0 到 m-1。设队头指针为 front，队尾指针是 rear，约定 front 指向队头元素的前一位置，rear 指向队尾元素。当 front 等于-1 时队空，rear 等于 m-1 时为队满。由于队列的性质（“删除”在队头而“插入”在队尾），所以当队尾指针 rear 等于 m-1 时，若 front 不等于-1，则队列中仍有空闲单元，所以队列并不是真满。这时若再有入队操作，会造成假“溢出”。其解决办法有二，一是将队列元素向前“平移”（占用 0 至 rear-front-1）；二是将队列看成首尾相连，即循环队列（0..m-1）。在循环队列下，仍定义 front=rear 时为队空，而判断队满则用两种办法，一是用“牺牲一个单元”，即 rear+1=front（准确记是（rear+1）%m=front，m 是队列容量）时为队满。另一种解法是“设标记”方法，如设标记 tag，tag 等于 0 情况下，若删除时导致 front=rear 为队空；tag=1 情况下，若因插入导致 front=rear 则为队满。

**四 算法设计题**

1. 解：方法是先依次让单链表上的元素进栈，然后再依次出栈。

```
Void invert (lklist head)
 {LstackTp s;
  initstack(s);
  p= head;
   while (p<>null)
     {Push (s,p->data);p=p->next;}
     p=head;
```

```
    while(not emptystack(s))
     {pop(s,p->data); p=p->next;}
  }
```

2. [题目分析]判断表达式中括号是否匹配，可通过栈，简单说是左括号时进栈，右括号时退栈。退栈时，若栈顶元素是左括号，则新读入的右括号与栈顶左括号就可消去。如此下去，输入表达式结束时，栈为空则正确，否则括号不匹配。

```
int EXYX(char E[],int n)
//E[]是有 n 字符的字符数组，存放字符串表达式，以‘#’结束。本算法判断表达式中圆括号是否匹配。
 {char s[30];          //s 是一维数组，容量足够大，用作存放括号的栈。
  int top=0;           //top 用作栈顶指针。
  s[top]= ‘#’;         //‘#’先入栈，用于和表达式结束符号‘#’匹配。
  int i=0;             //字符数组 E 的工作指针。
  while(E[i]!= ‘#’)    //逐字符处理字符表达式的数组。
    switch(E[i])
    {case ‘(’ :  s[++top]= ‘(’ ; i++ ;  break ;
     case ‘)’ :  if(s[top]== ‘(’ {top--; i++; break;}
                   else{printf(“括号不配对”);exit(0);}
     case ‘#’ :  if(s[top]== ‘#’ ){printf(“括号配对\n”);return (1);}
                 else {printf(“ 括号不配对\n”);return (0);} //括号不配对
     default :    i++;    //读入其它字符，不作处理。
    }
  }//算法结束。
```

[算法讨论]本题是用栈判断括号匹配的特例：只检查圆括号的配对。一般情况是检查花括号（‘{’，‘}’）、方括号（‘[’，‘]’）和圆括号（‘(’，‘)’）的配对问题。编写算法中如遇左括号（‘{’，‘[’，或‘(’）就压入栈中，如遇右括号（‘}’，‘]’，或‘)’），则与栈顶元素比较，如是与其配对的括号（左花括号，左方括号或左圆括号），则弹出栈顶元素；否则，就结论括号不配对。在读入表达式结束符‘#’时，栈中若应只剩‘#’，表示括号全部配对成功；否则表示括号不匹配。

另外，由于本题只是检查括号是否匹配，故对从表达式中读入的不是括号的那些字符，一律未作处理。再有，假设栈容量足够大，因此入栈时未判断溢出。

3. （1）A 和 D 是合法序列，B 和 C 是非法序列。

（2）设被判定的操作序列已存入一维数组 A 中。

```
int Judge(char A[])
 //判断字符数组 A 中的输入输出序列是否是合法序列。如是，返回 true，否则返回 false。
 {i=0;                  //i 为下标。
  j=k=0;                //j 和 k 分别为 I 和字母 O 的的个数。
  while(A[i]!= ‘\0’ ) //当未到字符数组尾就作。
    {switch(A[i])
      {case ‘I’ : j++; break; //入栈次数增 1。
       case ‘O’ : k++; if(k>j){printf(“序列非法\n”); exit(0);}
```

```
        }
      i++; //不论 A[i]是‘I’或‘0’，指针 i 均后移。}
    if(j!=k) {printf(“序列非法\n”)；return(false);}
    else {printf(“序列合法\n”)；return(true);}
   }//算法结束。
```

[算法讨论]在入栈出栈序列（即由‘I’和‘0’组成的字符串）的任一位置，入栈次数（‘I’的个数）都必须大于等于出栈次数（即‘0’的个数），否则视作非法序列，立即给出信息，退出算法。整个序列（即读到字符数组中字符串的结束标记‘\0’），入栈次数必须等于出栈次数（题目中要求栈的初态和终态都为空），否则视为非法序列。

4. [题目分析]两栈共享向量空间，将两栈栈底设在向量两端，初始时，s1 栈顶指针为-1，s2 栈顶为 maxsize。两栈顶指针相邻时为栈满。两栈顶相向，迎面增长，栈顶指针指向栈顶元素。

```
#define maxsize    两栈共享顺序存储空间所能达到的最多元素数
#define elemtp int        //假设元素类型为整型
typedef struct
{elemtp stack[maxsize];  //栈空间
 int top[2];             //top 为两个栈顶指针
}stk;
stk s;                   //s 是如上定义的结构类型变量，为全局变量。
```

(1)入栈操作：

```
int push(int i,int x)
//入栈操作。i 为栈号，i=0 表示左边的栈 s1，i=1 表示右边的栈 s2，x 是入栈元素。
入栈成功返回 1，否则返回 0。
{if(i<0||i>1){printf(“栈号输入不对”);exit(0);}
if(s.top[1]-s.top[0]==1) {printf(“栈已满\n”);return(0);}
switch(i)
 {case 0: s.stack[++s.top[0]]=x; return(1); break;
  case 1: s.stack[--s.top[1]]=x; return(1);
 }
}//push
```

（2） 退栈操作

```
elemtp pop(int i)
//退栈算法。i 代表栈号，i=0 时为 s1 栈，i=1 时为 s2 栈。退栈成功返回退栈元素，
 否则返回-1。
{if(i<0 || i>1){printf(“栈号输入错误\n”)；exit(0);}
  switch(i)
   {case 0: if(s.top[0]==-1) {printf(“栈空\n”)；return（-1)；}
            else return(s.stack[s.top[0]--]);
    case 1: if(s.top[1]==maxsize {printf(“栈空\n”); return(-1);}
            else return(s.stack[s.top[1]++]);
    }
}//算法结束
```

[算法讨论] 请注意算法中两栈入栈和退栈时的栈顶指针的计算。两栈共享空间示意图

略，s1 栈是通常意义下的栈，而 s2 栈入栈操作时，其栈顶指针左移（减 1），退栈时，栈顶指针右移（加 1）。

5. [题目分析]栈的特点是后进先出，队列的特点是先进先出。所以，用两个栈 s1 和 s2 模拟一个队列时，s1 作输入栈，逐个元素压栈，以此模拟队列元素的入队。当需要出队时，将栈 s1 退栈并逐个压入栈 s2 中，s1 中最先入栈的元素，在 s2 中处于栈顶。s2 退栈，相当于队列的出队，实现了先进先出。显然，只有栈 s2 为空且 s1 也为空，才算是队列空。

(1) int enqueue(stack s1,elemtp x)

//s1 是容量为 n 的栈，栈中元素类型是 elemtp。本算法将 x 入栈，若入栈成功返回 1，否则返回 0。

```
{if(top1==n && !Sempty(s2))        //top1 是栈 s1 的栈顶指针，是全局变量。
   {printf("栈满");return(0);}   //s1 满 s2 非空,这时 s1 不能再入栈。
if(top1==n && Sempty(s2))          //若 s2 为空，先将 s1 退栈,元素再压栈到 s2。
   {while(!Sempty(s1)) {POP(s1,x);PUSH(s2,x);}
 PUSH(s1,x); return(1);  //x 入栈，实现了队列元素的入队。
}
```

(2) void dequeue(stack s2,s1)

```
//s2 是输出栈，本算法将 s2 栈顶元素退栈，实现队列元素的出队。
  {if(!Sempty(s2))          //栈 s2 不空，则直接出队。
    {POP(s2,x);   printf("出队元素为",x);  }
   else                     //处理 s2 空栈。
     if(Sempty(s1)) {printf("队列空");exit(0);}//若输入栈也为空，则判定队空。
     else                   //先将栈 s1 倒入 s2 中，再作出队操作。
       {while(!Sempty(s1)) {POP(s1,x);PUSH(s2,x);}
        POP(s2,x);          //s2 退栈相当队列出队。
        printf("出队元素", x);
       }
 }//结束算法 dequue。
```

(3) int queue_empty()

```
//本算法判用栈 s1 和 s2 模拟的队列是否为空。
{if(Sempty(s1)&&Sempty(s2))  return(1);//队列空。
 else return(0);                        //队列不空。
}
```

[算法讨论]算法中假定栈 s1 和栈 s2 容量相同。出队从栈 s2 出，当 s2 为空时，若 s1 不空，则将 s1 倒入 s2 再出栈。入队在 s1，当 s1 满后，若 s2 空，则将 s1 倒入 s2，之后再入队。因此队列的容量为两栈容量之和。元素从栈 s1 倒入 s2，必须在 s2 空的情况下才能进行，即在要求出队操作时，若 s2 空，则不论 s1 元素多少（只要不空），就要全部倒入 s2 中。

6.【解答】入队算法：

```
int  EnterQueue(SeqQueue  *Q,  QueueElementType  x)
{  /*将元素 x 入队*/
   if(Q->front==Q->rear  &&  tag==1)     /*队满*/
      return(FALSE);
```

```
        if(Q->front==Q-> rear  &&  tag==0)   /*x 入队前队空，x 入队后重新设置标志
*/
           tag=1;
        Q->elememt[Q->rear]=x;
        Q->rear=(Q->rear+1)%MAXSIZE;      /*设置队尾指针*/
        Return(TRUE);
    }
```

出队算法：

```
int  DeleteQueue( SeqQueue  *Q ,  QueueElementType  *x)
{ /*删除队头元素，用 x 返回其值*/
  if(Q->front==Q->rear  &&  tag==0)     /*队空*/
    return(FALSE);
  *x=Q->element[Q->front];
  Q->front=(Q->front+1)%MAXSIZE;    /*重新设置队头指针*/
  if(Q->front==Q->rear)  tag=0;     /*队头元素出队后队列为空，重新设置标志域
*/
  Return(TUUE);
}
```

7.［题目分析］根据队列先进先出和栈后进先出的性质，先将非空队列中的元素出队，并压入初始为空的栈中。这时栈顶元素是队列中最后出队的元素。然后将栈中元素出栈，依次插入到初始为空的队列中。栈中第一个退栈的元素成为队列中第一个元素，最后退栈的元素（出队时第一个元素）成了最后入队的元素，从而实现了原队列的逆置。

```
void Invert(queue Q)
//Q 是一个非空队列，本算法利用空栈 S 和已给的几个栈和队列的 ADT 函数，将队列 Q 中
的元素逆置。
  {makempty(S); //置空栈
   while (!isEmpty(Q))  // 队列 Q 中元素出队
   {value=deQueue(Q);  push(S,value);  }// 将出队元素压入栈中
  while(!isEmpty(S)) //栈中元素退栈
    {value=pop(S);   enQueue(Q,value); }//将出栈元素入队列 Q
   }//算法 invert 结束
```

# 第四章　串

## 一、单项选择题

1. B
2. B
3. B
4. C
5. C

## 二、填空题

1. 空、字符
2.　由空格字符（ASCII 值 32）所组成的字符串　　空格个数
3. 长度、相等、子、主
4. 5
5. 01122312
6.（1）char s[ ]　　(2) j++　　(3) i >= j
7. [题目分析]本题算法采用顺序存储结构求串 s 和串 t 的最大公共子串。串 s 用 i 指针（1<=i<=s.len）。t 串用 j 指针（1<=j<=t.len）。算法思想是对每个 i（1<=i<=s.len，即程序中第一个 WHILE 循环），来求从 i 开始的连续字符串与从 j（1<=j<=t.len，即程序中第二个WHILE 循环）开始的连续字符串的最大匹配。程序中第三个（即最内层）的 WHILE 循环，是当 s 中某字符（s [i]）与 t 中某字符（t [j]）相等时，求出局部公共子串。若该子串长度大于已求出的最长公共子串（初始为 0 ），则最长公共子串的长度要修改。

   (1) i+k<=s.len && j+k<=t.len && s[i+k]==t[j+k]　//所有注释同上（a）
   (2) con=0　(3) j+=k　　(4) j++　　(5) i++

## 三、应用题

1. 空格是一个字符，其 ASCII 码值是 32。空格串是由空格组成的串，其长度等于空格的个数。空串是不含任何字符的串，即空串的长度是零。

2. (a)A+B　　“　mule”
   (b)B+A　　“mule　”
   (c)D+C+B　　“myoldmule”
   (d)SUBSTR(B,3,2)　　“le”
   (e)SUBSTR(C,1,0)　　“ ”
   (f)LENGTH(A)　　2
   (g)LENGTH(D)　　2
   (h)INDEX(B,D)　　0
   (i)INDEX(C,”d”)　　3
   (j)INSERT(D,2,C)　　“myold”
   (k)INSERT(B,1,A)　　“m　ule”
   (l)DELETE(B,2,2)　　“me”
   (m)DELETE(B,2,0)　　“mule”

3. 朴素的模式匹配（Brute－Force）时间复杂度是 O（m＊n），KMP 算法有一定改进，时间

复杂度达到O（m＋n）。本题也可采用从后面匹配的方法，即从右向左扫描，比较6次成功。另一种匹配方式是从左往右扫描，但是先比较模式串的最后一个字符，若不等，则模式串后移；若相等，再比较模式串的第一个字符，若第一个字符也相等，则从模式串的第二个字符开始，向右比较，直至相等或失败。若失败，模式串后移，再重复以上过程。按这种方法，本题比较18次成功。

4．next和nextval值分别为0112123422和0102010422。

5．题中所给操作的含义如下：

//：连接函数，将两个串连接成一个串

substr（s,i,j)：取子串函数，从串s的第i个字符开始，取连续j个字符形成子串

replace（s1,i,j,s2)：置换函数，用s2串替换s1串中从第i个字符开始的连续j个字符

本题有多种解法，下面是其中的一种：

（1） s1=substr（s,3,1）　　//取出字符：‘y’

（2） s2=substr（s,6,1）　　//取出字符：‘+’

（3） s3=substr（s,1,5）　　//取出子串：‘(xyz)’

（4） s4=substr（s,7,1）　　//取出字符：‘*’

（5） s5=replace（s3,3,1,s2）//形成部分串：‘(x+z)’

（6） s=s5//s4//s1　　//形成串t即‘(x+z）*y’

**四、算法设计题**

1．const maxlen=串的最大长度;

```
typedef struct
  {char ch [maxlen];
   int curlen;
} string;
int EQUAL_string(string s,string t )
{ if (s.curlen!=t.curlen) return(0);
 for (t=0; t<s.curlen; t++)
  if (s.ch[t]!=t.ch[t] ) return(0);
  return(1);
}
```

2．设单链表无头结点

```
const  nodesize =用户定义的结点大小;
typedef struct node *pointer;
struct node
  {char ch;
  pointer next;
  }
typedef pointer strlist;
int     EQULA_strlist(strlist s ,strlist t )
{  while ((s!= null )&&(t!=null)&&(s->ch==t->ch))
```

```
   {s=s->next;t=t->next;}
   return((t= = null)&&(s= =null));
}
```

3. 分析：首先判断串 T 是否为串 S 的子串，若串 T 是串 S 的子串对 S 中该子串逆置。

```
 Int NZ_strlist (strlist s,strlist t)
  {p=s->next;t=t->next;q=s;
  while(p!=null)
 {pp =p ; tt =t;  /*判断串 T 是否为串 S 的子串*/
 while ((tt!=null)&&(pp!=null)&&(pp->ch= =tt->ch))
  {pp=pp->next;tt=tt->next;}
if (tt==null)  /*串 T 是串 S 的子串对 S 中的该子串的位置*/
 {qq=q->next; /* q 是子串的第一个结点前趋 pp 是子串最后一个结点后继*/
 while(qq!=pp)
 {g=qq; qq= qq->next;
  q->next =pp; pp=g;
 }
 q->next=pp;    /*将该子串的前趋与逆置后的子串的相连*/
 return(1);/*找到并逆置返 1 */
}else {q=p;p=p->next;}
}
return(0);/*找不到匹配的串返 0*/
}
```

4. [题目分析]设以字符数组 s 表示串，重复子串的含义是由一个或多个连续相等的字符组成的子串，其长度用 max 表示，初始长度为 0，将每个局部重复子串的长度与 max 相比，若比 max 大，则需要更新 max，并用 index 记住其开始位置。

```
int LongestString(char s[],int n)
//串用一维数组 s 存储，长度为 n，本算法求最长重复子串，返回其长度。
{int index=0,max=0;  //index 记最长的串在 s 串中的开始位置，max 记其长度
 int length=1,i=0,start=0;   //length 记局部重复子串长度，i 为字符数组下标
 while(i<n-1)
   if(s[i]==s[i+1]) {i++; length++;}
   else //上一个重复子串结束
      {if(max<length) {max=length; index=start; } //当前重复子串长度大，则
更新 max
       i++;start=i;length=1;                    //初始化下一重复子串的起始
位置和长度
      }
 printf("最长重复子串的长度为%d，在串中的位置%d\n"，max,index);
 return(max);
}//算法结束
```

[算法讨论]算法中用 i<n-1 来控制循环次数，因 C 数组下标从 0 开始，故长度为 n 的串，其最后一个字符下标是 n-1，当 i 最大为 n-2 时，条件语句中 s[i+1]正好是 s[n-1]，

即最后一个字符。子串长度的初值数为 1，表示一个字符自然等于其身。

算法的时间复杂度为 O(n)，每个字符与其后继比较一次。

5. [题目分析]实现字符串的逆置并不难，但本题“要求不另设串存储空间”来实现字符串逆序存储，即第一个输入的字符最后存储，最后输入的字符先存储，使用递归可容易做到。

```
 void InvertStore(char A[])
//字符串逆序存储的递归算法。
{ char ch;
  static int i = 0;//需要使用静态变量
  scanf ("%c",&ch);
  if (ch!= '.')    //规定'.'是字符串输入结束标志
    {InvertStore(A);
     A[i++] = ch;//字符串逆序存储
    }
  A[i] = '\0';  //字符串结尾标记
}//结束算法 InvertStore。
```

# 第 5 章　数组和广义表

## 一、单项选择题

1.　C
2.　C
3.　B
4.　A
5.　C
6.　A
7.　A
8.　C
9.　C
10.　A
11.　C

## 二、填空题

1. 顺序、列序、行序
2. 第 1 行　　第 3 列
3. i(i-1)/2+j (1<=i,j<=n)
4. 非零元很少(t<<m*n)且分布没有规律
5.　（1） 原子（单元素）是结构上不可再分的，可以是一个数或一个结构；而表带结构，本质就是广义表，因作为广义表的元素故称为子表。
　（2）大写字母　（3）小写字母　（4）表中元素的个数（5）表展开后所含括号的层数
6.（1）（）　（2）(()）　（3）2　（4）2
7.　col<=a.nu,　a.data[p].j,　q++
8.　（1）(p->tag==0)　　//处理原子
（2）h=reverse(p->val.ptr.hp)　//处理表头
（3）(p->val.ptr.tp)　　//产生表尾的逆置广义表
（4）s->val.ptr.tp=t;　　//连接
（5）q->val.ptr.hp=h　　//头结点指向广义表

## 三、应用题

1. 958　三维数组以行为主序存储，其元素地址公式为：

$$LOC(A_{ijk})=LOC(A_{c1c2c3})+[(i-c_1)V_2V_3+(j-c_2)V_3+(k-c_3)]*L+1$$

其中 $c_i$,$d_i$ 是各维的下界和上界，$V_i=d_i-c_i+1$ 是各维元素个数，L 是一个元素所占的存储单元数。

2. 特殊矩阵指值相同的元素或零元素在矩阵中的分布有一定规律，因此可以对非零元素分配单元（对值相同元素只分配一个单元），将非零元素存储在向量中，元素的下标 i 和 j 和该元素在向量中的下标有一定规律，可以用简单公式表示，仍具有随机存取功能。而稀疏矩阵是指非零元素和矩阵容量相比很小（t<<m*n），且分布没有规律。用十字链表作存储结构自然失去了随机存取的功能。即使用三元组表的顺序存储结构，存取下标为 i 和 j 的元素时，要扫描三元组表，下标不同的元素，存取时间也不同，最好情况下存取时间为 O(1)，最差情况下是 O(n)，因此也失去了随机存取的功能。

3. 数组是具有相同性质的数据元素的集合，同时每个元素又有唯一下标限定，可以说数组是值和下标偶对的有限集合。n 维数组中的每个元素，处于 n 个关系之中，每个关系都是线性的，且 n 维数组可以看作其元素是 n-1 维数组的一个线性表。广义表中的元素，可以是原子，也可以是子表，即广义表是原子或子表的有限序列，满足线性结构的特性：在非空线性结构中，只有一个称为“第一个”的元素，只有一个成为“最后一个”的元素，第一元素有后继而没有前驱，最后一个元素有前驱而没有后继，其余每个元素有唯一前驱和唯一后继。从这个意义上说，广义表属于线性结构。

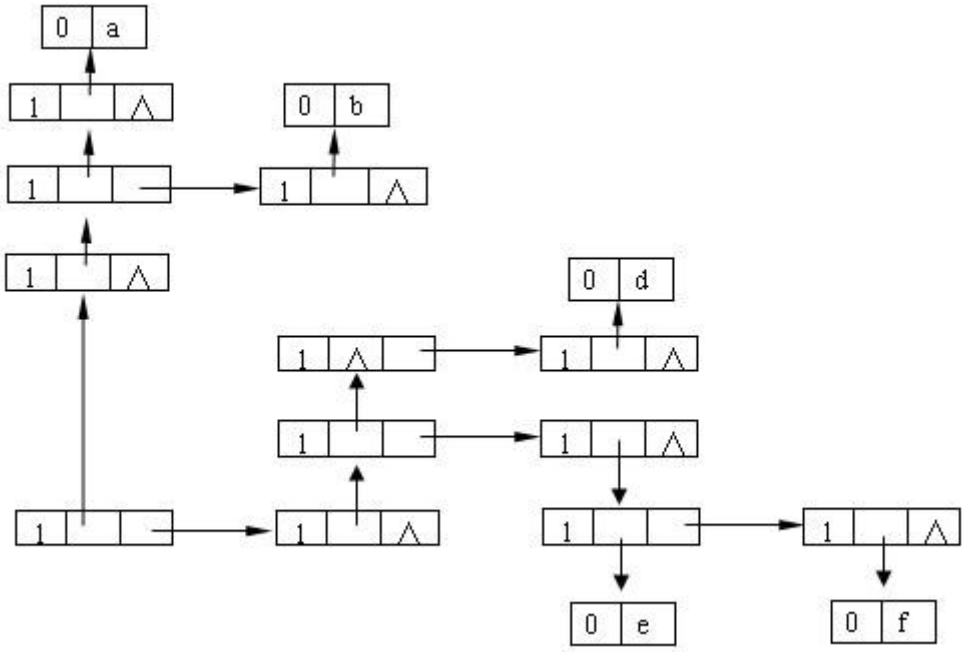
4. 三对角矩阵第一行和最后一行各有两个非零元素，其余每行均有三个非零元素，所以共有 3n-2 个元素。

（1）主对角线左下对角线上的元素下标间有 i=j+1 关系，k 与 i 和 j 的关系为 k=3(i-1)；主对角线上元素下标间有关系 i=j，k 与 i 和 j 的关系为 k=3(i-1)+1；主对角线右上那条对角线上元素下标间有关系 i=j-1，k 与 i 和 j 的关系为 k=3(i-1)+2。综合以上三等式，有 k=2(i-1)+j　(1<=i,j<=n, |i-j|<=1)

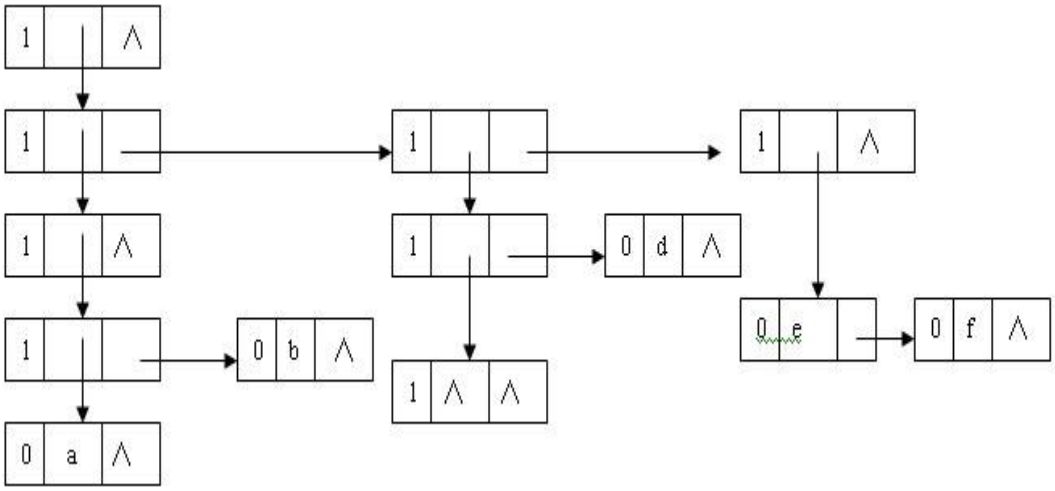
（2）i=k/3+1；（1≤k≤3n-2）　// k/3 取小于 k/3 的最大整数。下同

j=k-2(i-1)=k-2(k/3)=k%3+k/3

5.




第一种存储结构




第二种存储结构

6.

（1） HEAD[((a,b),(c,d))]; (a,b)

（2） TAIL[((a,b),(c,d))]; ((c,d))

（3） TAIL[HEAD[((a,b),(c,d))]]; (b)

（4） HEAD[TAIL[HEAD[((a,b),(c,d))]]]; b

（5） TAIL[HEAD[TAIL[((a,b),(c,d))]]]; (d)

7. Head（Tail（Head（Head（L1））））

Head（Head（Head（Tail（Head（Tail（L2））））））

**四、算法设计题**

1. . [题目分析]矩阵中元素按行和按列都已排序，要求查找时间复杂度为 O（m+n），因此不能采用常规的二层循环的查找。可以先从右上角（i=a,j=d）元素与 x 比较，只有三种情况：一是 A[i,j]>x， 这情况下向 j 小的方向继续查找；二是 A[i,j]<x，下步应向 i 大的方向查找；三是 A[i,j]=x，查找成功。否则，若下标已超出范围，则查找失败。

```
void search(datatype A[ ][ ], int a,b,c,d, datatype x)
 //n*m 矩阵 A,行下标从 a 到 b,列下标从 c 到 d,本算法查找 x 是否在矩阵 A 中.
 {i=a; j=d; flag=0; //flag 是成功查到 x 的标志
  while(i<=b && j>=c)
    if(A[i][j]==x) {flag=1;break;}
    else if (A[i][j]>x) j--; else i++;
  if(flag) printf("A[%d][%d]=%d",i,j,x);      //假定 x 为整型.
  else printf("矩阵 A 中无%d 元素", x);
 }算法 search 结束。
```

[算法讨论]算法中查找 x 的路线从右上角开始，向下（当 x>A[i,j]）或向左（当 x<A[i,j]）。向下最多是 m,向左最多是 n。最佳情况是在右上角比较一次成功,最差是在左下角(A[b,c])，比较 m+n 次，故算法最差时间复杂度是 O(m+n)。

2.[题目分析]判断二维数组中元素是否互不相同，只有逐个比较,找到一对相等的元素，就可结论为不是互不相同。如何达到每个元素同其它元素比较一次且只一次？在当前行，每个元素要同本行后面的元素比较一次（下面第一个循环控制变量 p 的 for 循环），然后同第 i+1

行及以后各行元素比较一次，这就是循环控制变量 k 和 p 的二层 for 循环。

```
int JudgEqual(ing a[m][n],int m,n)
 //判断二维数组中所有元素是否互不相同，如是，返回 1；否则，返回 0。
{for(i=0;i<m;i++)
  for(j=0;j<n-1;j++)
   { for(p=j+1;p<n;p++) //和同行其它元素比较
      if(a[i][j]==a[i][p]) {printf(“no”); return(0); }
  //只要有一个相同的，就结论不是互不相同
     for(k=i+1;k<m;k++)  //和第 i+1 行及以后元素比较
      for(p=0;p<n;p++)
         if(a[i][j]==a[k][p]) {printf(“no”); return(0); }
}// for(j=0;j<n-1;j++)
printf(yes”); return(1);   //元素互不相同
}//算法 JudgEqual 结束
```

（2）二维数组中的每一个元素同其它元素都比较一次，数组中共 m*n 个元素，第 1 个元素同其它 m*n-1 个元素比较，第 2 个元素同其它 m*n-2 个元素比较，……，第 m*n-1 个元素同最后一个元素(m*n)比较一次，所以在元素互不相等时总的比较次数为(m*n-1)+(m*n-2)+…+2+1=（m*n）(m*n-1)/2。在有相同元素时,可能第一次比较就相同,也可能最后一次比较时相同,设在(m*n-1)个位置上均可能相同,这时的平均比较次数约为（m*n）(m*n-1)/4，总的时间复杂度是 $O(n^4)$。

3. [题目分析]题目要求按 B 数组内容调整 A 数组中记录的次序，可以从 i=1 开始，检查是否 B[i]=i。如是，则 A[i]恰为正确位置，不需再调；否则，B[i]=k≠i，则将 A[i]和 A[k]对调，B[i]和 B[k]对调，直到 B[i]=i 为止。

```
void CountSort (rectype A[],int B[])
//A 是 100 个记录的数组，B 是整型数组，本算法利用数组 B 对 A 进行计数排序
{int i,j,n=100;
i=1;
while(i<n)
 {if(B[i]!=i)  //若 B[i]=i 则 A[i]正好在自己的位置上，则不需要调整
   { j=i;
     while (B[j]!=i)
       { k=B[j]; B[j]=B[k]; B[k]=k;    // B[j]和 B[k]交换
         r0=A[j];A[j]=A[k]; A[k]=r0; } //r0 是数组 A 的元素类型,A[j]和 A[k]
交换
   i++;} //完成了一个小循环，第 i 个已经安排好
 }//算法结束
```

4. 解答：本题首先判断三元组 A，B 表示的矩阵是否行列相同，若相同才能进行矩阵的加法运算。若三元组表示的矩阵能进行相加运算，其思路如下：

若 a,b 表的指针均没有到表尾，重复下列步骤：

（1） 若 a 表元素的 i 域值小于 b 表元素的 i 域值，将 a 表当前元素插入到 c 表表尾， a 表指针后移。

（2） 若 a 表元素的 i 域值大于 b 表元素的 i 域值，将 b 表当前元素插入到 c 表表尾， b

表指针后移。

（3） 若 a 表元素的 i 域值等于 b 表元素的 i 域值，又分以下几种情况讨论：

①若 a 表元素的 j 域值小于 b 表元素的 j 域值，将 a 表当前元素插入到 c 表表尾，a 表指针后移。

②若 a 表元素的 j 域值大于 b 表元素的 j 域值，将 b 表当前元素插入到 c 表表尾，b 表指针后移。

③若 a 表元素的 j 域值等于 b 表元素的 j 域值，若 a,b 表当前元素的 v 域值和非零，则在 c 表表尾播入元素的 I\j 域值等于 a 表当前元素的 I\j 域 的值，域 v 的值等于 a,b 表的域值的和，将 a,b 表当前指针后移。

(4) 若 a 表的指针没到表尾，b 表的指针到表尾，将 a 表剩余元素依次插入到 c 表表尾，否则，将 b 表剩余元素依次插入到 c 表表尾.

```
SpMaterixTp trsxsum(SpMaterixTp a, SpMaterixTp b , SpMaterixTp c)
{if ( (a .mu=b. mu)&&(a.nu=b.mu))                /*a，b 为同行同列矩阵*/
  {c.mu=a.mu ;c.nu=a.nu; I=1; j=1; p=0;
    if (a.tu&&b.tu)                              /*a，b 为非空矩阵*/
      {cola=a.data[I].i; rowa = a.data[I] .j;  /*取三元组 a 的元素的行、列下标*/
       colb= b.data[j].i; rowb =b.data[j] .j;  /*取三元组 b 的元素的行、列下标*/
      while ((I<=a.tu)&&(j<=b.tu))
         switch
            {cola <colb:     /*在 c 中，插入三元组 a 的元素*/
               p++; c.data [p] .i=a.data [I] .i;
               c.data [p].j= a.data [I].j;
               c.data [p] .v= a.data[I].v;
               I++; break;   /*a 元素后移*/
             Cola>colb:      /*在 c 中，插入三元组 b 的元素*/
              P++;c.data[p] .i=b.data [j] .i;
               c.data [p].j= b.data [j].j;
               c.data [p] .v= b.data[j].v;
               j++; break;   /*b 元素后移*/
             Cola =colb :
               P++;          /*三元组 a，b 的元素 I 域相等*/
               If (rowa<rowb)             /*在 c 中插入三元组 a 的元素*/
                 {c.data[p].j=a.data [I].j;
                  c.data[p].j=a.data[j].j;
                  c.data[p].v=a.data[I].v; I++;
                  }
                  else if (rowa>rowb)  /*在 c 中插入三元组 b 的元素*/
                      {c.data[p].i=b.data[j].i;
                       c.data[p].j=b.data[j].j;
                       c.data[p].v= b.data[j].v; j++;
                       }
                  else if (a.data[I].v +b.data[j].v)
                    /*a 中的元素和 b 中的元素 I，j 的域都相等且 v 域的和非零*/
                   {c.data [p].i =a.data[I].i ;/*在 c 中元素*/
```

```
                    c.data[p].j=a.data[j].j;
                    c.data[p].v=a.data[I].v+b.data[j].v;
                    I++; j++;
                  } else {p--;I++;j++};  /*a 中元素和 b 中元素的 I，j 域都相等且 v
                                                    域的和为零*/
            break;
        }   /*swith 结束*/
}
if (I<=a.tu)                    /*b 表到表尾，将 a 表中剩余元素插入到 C 表中*/
  {p++; c.data[p].i =a.data[p].i; c.data[p].j=a.data[p].j;
   c.data[p].v=a.data[p].v; I++;
   else{                    /*a 表到表尾，将 b 表中剩余元素插入到 C 表中*/
         p++; c.data[p].i=b.data[j].i; c.data[p].j=b.data[j].j;
         c.data[p].v=b.data[j].v; j++;}
}c.tu=p;                        /*c 表赋非零元素个数*/
}
```

5. 、[题目分析] 广义表的元素有原子和表。在读入广义表“表达式”时，遇到左括号‘(’就递归的构造子表，否则若是原子，就建立原子结点；若读入逗号‘,’，就递归构造后续子表；若 n=0，则构造含空格字符的空表，直到碰到输入结束符号（‘#’）。设广义表的形式定义如下：

```
    typedef  struct node
    {int tag;                   //tag=0 为原子，tag=1 为子表
     struct node *link;         //指向后继结点的指针
     union {struct node *slink; //指向子表的指针
            char data;          //原子
            }element;
    }Glist;
    Glist *creat ()
    //建立广义表的存储结构
    {char ch; Glist *gh;
     scanf(“%c”,&ch);
     if(ch==’ ’ ) gh=null;
       else {gh=(Glist*)malloc(sizeof(Glist));
             if(ch== ‘(’ ){gh->tag=1; //子表
                           gh->element.slink=creat(); } //递归构造子表
            else {gh->tag=0;gh->element.data=ch;} //原子结点
            }
     scanf(“%c”,&ch);
     if(gh!=null) if(ch== ‘,’ ) gh->link=creat();  //递归构造后续广义表
                  else gh->link=null;
     return(gh);
     }
  }算法结束
```

# 第六章　树和二叉树

## 一、单项选择题

1.A
2.D
3．A
4．C
5．B
6．D
7．E
8．D
9．C
10．B
11．C
12．A
13．D
14．B
15．C
16．B
17．D
18．B
19．D
20．C

## 二、判断题（在各题后填写“√”或“×”）

1．完全二叉树一定存在度为 1 的结点。×
2．对于有 N 个结点的二叉树，其高度为 $\log_2 n$。×
3．二叉树的遍历只是为了在应用中找到一种线性次序。√
4．一棵一般树的结点的前序遍历和后序遍历分别与它相应二叉树的结点前序遍历和后序遍历是一致的。×
5．用一维数组存储二叉树时，总是以前序遍历顺序存储结点。×
6．中序遍历一棵二叉排序树的结点就可得到排好序的结点序列　√
7．完全二叉树中，若一个结点没有左孩子，则它必是树叶。√
8．二叉树只能用二叉链表表示。×
9．给定一棵树，可以找到唯一的一棵二叉树与之对应。√
10．用链表(llink-rlink)存储包含 n 个结点的二叉树，结点的 2n 个指针区域中有 n-1 个空指针。×
11．树形结构中元素之间存在一个对多个的关系。√
12．将一棵树转成二叉树，根结点没有左子树。×
13．度为二的树就是二叉树。×
14．二叉树中序线索化后，不存在空指针域。×
15．霍夫曼树的结点个数不能是偶数。√
16．哈夫曼树是带权路径长度最短的树，路径上权值较大的结点离根较近。√

**三、填空题**

1. p->lchild==null && p->rchlid==null

2.(1)2k-1　(2)2k-1

3. 64

4. 2n　n-1　n+1

5. 先序遍历　后序遍历　中序遍历

6..(1)2k-2+1（第 k 层 1 个结点，总结点个数是 2H-1，其双亲是 2H-1/2=2k-2）(2)⌊log2i⌋+1

7. 4

8. 任何结点至多只有右子女的二叉树。

9. 前序

10. 69

11. count++,　countleaf(l->rchild,count)

12. (1) p=p->lchild　//　沿左子树向下　（2）p=p->rchild

13. (1)0　（2）hl>hr　(3)hr=hl

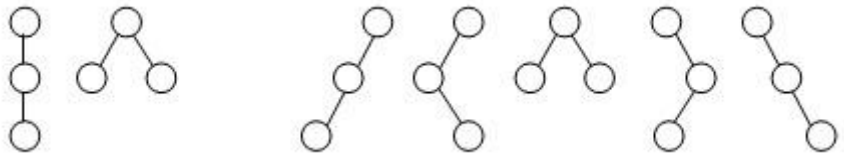
14. (1)p->rchild　(2)p->lchild　(3)p->lchild
(4)ADDQ(Q,p->lchild)　(5)ADDQ(Q,p->rchild)

**四、应用题**

1. 树和二叉树逻辑上都是树形结构，树和二叉树的区别有三：一是二叉树的度至多为 2，树无此限制；二是二叉树有左右子树之分，即使在只有一个分枝的情况下，　也必须指出是左子树还是右子树，树无此限制；三是二叉树允许为空，树一般不允许为空（个别书上允许为空）。二叉树不是树的特例。

2.【解答】

具有 3 个结点的树　　具有 3 个结点的二叉树



3. 解答：先根序列：A B C D E F G ；
中根序列：B C D A F E ；
后根序列：D C B F E A。

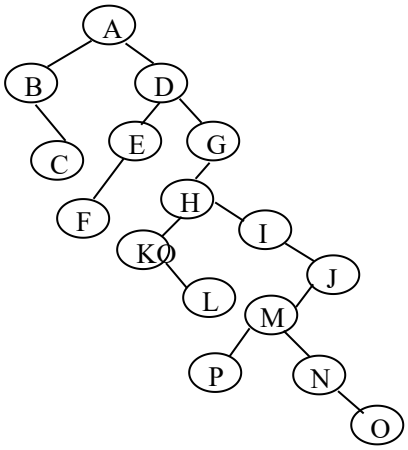
4. (1)$k^{h-1}$(h 为层数)

（2）因为该树每层上均有 $K^{h-1}$ 个结点，从根开始编号为 1，则结点 i 的从右向左数第 2 个孩子的结点编号为 ki。设 n 为结点 i 的子女，则关系式(i-1)k+2<=n<=ik+1 成立，因 i 是整数，故结点 n 的双亲 i 的编号为⌊n-2)/k⌋+1。

(3) 结点 n(n>1)的前一结点编号为 n-1（其最右边子女编号是(n-1)*k+1)，故结点 n 的第 i 个孩子的编号是(n-1)*k+1+i。

(4) 根据以上分析，结点 n 有右兄弟的条件是，它不是双亲的从右数的第一子女，即(n-1)%k!=0，其右兄弟编号是 n+1。

5.




6.（1）图略；

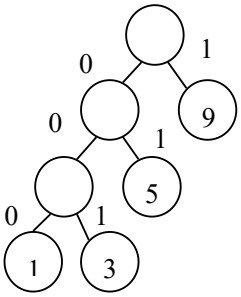
（2） 前序序列：A B C E D F H G I J

中序序列： E C B H F D J I G A

后序序列： E C H F J I G D B A

（3）图略。

7. 字符 A，B，C，D 出现的次数为 9,1,5,3。其哈夫曼编码如下 A:1，B:000，C:01，D:001




**五、算法设计题**

1. [题目分析]二叉树是递归定义的，以递归方式建立最简单。判定是否是完全二叉树，可以使用队列，在遍历中利用完全二叉树“若某结点无左子女就不应有右子女”的原则进行判断。

```
BiTree Creat()        //建立二叉树的二叉链表形式的存储结构
{ElemType x; BiTree bt;
  scanf(“%d”,&x);  //本题假定结点数据域为整型
  if(x==0) bt=null;
  else if(x>0)
         {bt=(BiNode *)malloc(sizeof(BiNode));
          bt->data=x; bt->lchild=creat(); bt->rchild=creat();
         }
        else error(“输入错误”);
  return(bt);
 }//结束 BiTree
int JudgeComplete(BiTree bt)  //判断二叉树是否是完全二叉树,如是，返回 1，否则，返回 0
```

```
{int tag=0; BiTree p=bt, Q[]; // Q 是队列，元素是二叉树结点指针，容量足够大
 if(p==null) return (1);
 QueueInit(Q); QueueIn(Q,p);  //初始化队列，根结点指针入队
 while (!QueueEmpty(Q))
   {p=QueueOut(Q);                                  //出队
    if (p->lchild && !tag) QueueIn(Q,p->lchild);  //左子女入队
    else {if (p->lchild) return 0;                //前边已有结点为空，本结点不空
          else tag=1;                              //首次出现结点为空
    if (p->rchild && !tag) QueueIn(Q,p->rchild);  //右子女入队
    else if (p->rchild) return 0; else tag=1;
    } //while
 return 1;  } //JudgeComplete
```

[算法讨论]完全二叉树证明还有其它方法。判断时易犯的错误是证明其左子树和右子数都是完全二叉树，由此推出整棵二叉树必是完全二叉树的错误结论。

2.[题目分析]后序遍历最后访问根结点，即在递归算法中，根是压在栈底的。采用后序非递归算法，栈中存放二叉树结点的指针，当访问到某结点时，栈中所有元素均为该结点的祖先。本题要找 p 和 q 的最近共同祖先结点 r，不失一般性，设 p 在 q 的左边。后序遍历必然先遍历到结点 p，栈中元素均为 p 的祖先。将栈拷入另一辅助栈中。再继续遍历到结点 q 时，将栈中元素从栈顶开始逐个到辅助栈中去匹配，第一个匹配（即相等）的元素就是结点 p 和 q 的最近公共祖先。

```
typedef struct
  {BiTree t;int tag;//tag=0  表示结点的左子女已被访问，tag=1 表示结点的右子女已被访问
  }stack;
stack s[],s1[];//栈，容量够大
BiTree Ancestor(BiTree ROOT,p,q,r)//求二叉树上结点 p 和 q 的最近的共同祖先结点 r。
 {top=0; bt=ROOT;
  while(bt!=null ||top>0)
  {while(bt!=null && bt!=p && bt!=q)              //结点入栈
    {s[++top].t=bt; s[top].tag=0; bt=bt->lchild;} //沿左分枝向下
   if(bt==p)  //不失一般性，假定 p 在 q 的左侧,遇结点 p 时，栈中元素均为 p 的祖先结点
     {for(i=1;i<=top;i++) s1[i]=s[i]; top1=top; }//将栈 s 的元素转入辅助栈 s1 保存
   if(bt==q)  //找到 q 结点。
      for(i=top;i>0;i--)//；将栈中元素的树结点到 s1 去匹配
        {pp=s[i].t;
         for (j=top1;j>0;j--)
          if(s1[j].t==pp) {printf("p 和 q 的最近共同的祖先已找到");return (pp);}
        }
    while(top!=0 && s[top].tag==1) top--;      //退栈
```

```
    if (top!=0) {s[top].tag=1;bt=s[top].t->rchild;}  //沿右分枝向下遍历
   }//结束 while(bt!=null ||top>0)
   return(null);// q 、p 无公共祖先
  } //结束 Ancestor
```

3. 解答：本算法要借用队列来完成，其基本思想是，只要队列不为空，就出队列，然后判断该结点是否有左孩子和右孩子，如有就依次输出左、右孩子的值，然后让左、右孩子进队列。

```
    void layorder (bitreptr T)
     {initqueue (q)                              /*队列初始化*/
      if(T!=NULL)
        {printf(“%f”, T->data);
         enqueue (q, T);                         /*入队列*/
         while (not emptyqueue (q) )             /*若队列非空*/
           {outqueue (q, p) ;                    /*出队*/
            if (p->lchild!=NULL)
               {printf(“%f”, p->lchild->data);
                enqueue (q, p->lchild);           /*入队列*/
               }
            if (p->rchild!=NULL)
              {printf(“%”, p->rchild->data);
               enqueue (q, p->rchild);            /*入队列*/
              }
           }
        }
 }
```

4.【解答】

```
    Void  PreOrder(BiTree  root)    /*先序遍历二叉树的非递归算法*/
    {
      InitStack(&S);
      p=root;
      while(p!=NULL || !IsEmpty(S) )
      { if(p!=NULL)
        {
          Visit(p->data);
          push(&S,p);
          p=p->Lchild;
        }
       else
        {
          Pop(&S,&p);
          p=p->RChild;
       }
     }
```

```
   }

5.. void InOrder(BiTree bt)
    {BiTree s[],p=bt;  //s 是元素为二叉树结点指针的栈，容量足够大
     int top=0;
     while(p || top>0)
        {while(p) {s[++top]=p; bt=p->lchild;}              //中序遍历左子树
         if(top>0){p=s[top--]; printf(p->data); p=p->rchild;} //退栈，访问，转右
子树
       }   }


6. BiTree  Copy(BiTree  t)//复制二叉树 t
   {BiTree bt;
    if (t==null) bt=null;
    else{bt=(BiTree)malloc(sizeof(BiNode)); bt->data=t->data;
         bt->lchild=Copy(t->lchild);
         bt->rchild=Copy(t->rchild);
         }
    return(bt); }//结束 Copy
```

7.[题目分析]叶子结点只有在遍历中才能知道，这里使用中序递归遍历。设置前驱结点指针 pre，初始为空。第一个叶子结点由指针 head 指向，遍历到叶子结点时，就将它前驱的 rchild 指针指向它，最后叶子结点的 rchild 为空。

```
    LinkedList head,pre=null;  //全局变量
    LinkedList InOrder(BiTree bt)
      //中序遍历二叉树 bt，将叶子结点从左到右链成一个单链表，表头指针为 head
     {if(bt){InOrder(bt->lchild);          //中序遍历左子树
             if(bt->lchild==null && bt->rchild==null) //叶子结点
                if(pre==null) {head=bt; pre=bt;} //处理第一个叶子结点
                else{pre->rchild=bt; pre=bt; }   //将叶子结点链入链表
             InOrder(bt->rchild);                 //中序遍历左子树
             pre->rchild=null;                    //设置链表尾
            }
        return(head);  } //InOrder
```

时间复杂度为 O(n),辅助变量使用 head 和 pre,栈空间复杂度 O(n)

8.[题目分析] 删除以元素值 x 为根的子树，只要能删除其左右子树，就可以释放值为 x 的根结点，因此宜采用后序遍历。删除值为 x 结点，意味着应将其父结点的左(右)子女指针置空，用层次遍历易于找到某结点的父结点。本题要求删除树中每一个元素值为 x 的结点的子树，因此要遍历完整棵二叉树。

```
    void  DeleteXTree(BiTree  bt)  //删除以 bt 为根的子树
     {DeleteXTree(bt->lchild); DeleteXTree(bt->rchild);//删除 bt 的左子树、右子
```

```
树
  free(bt);  }// DeleteXTree                          //释放被删结点所占的存储
空间
void  Search(B:Tree bt,ElemType x)
  //在二叉树上查找所有以 x 为元素值的结点，并删除以其为根的子树
 {BiTree Q[];//Q 是存放二叉树结点指针的队列，容量足够大
  if(bt)
   {if(bt->data==x) {DeleteXTree(bt); exit(0);}//若根结点的值为 x，则删除整
棵树
   {QueueInit(Q); QueueIn(Q,bt);
    while(!QueueEmpty(Q))
      {p=QueueOut(Q);
       if(p->lchild)                // 若左子女非空
         if(p->lchild->data==x)     //左子女结点值为 x,应删除当前结点的左子
树
          {DeleteXTree(p->lchild); p->lchild=null;} //父结点的左子女置空
         else Enqueue (Q,p->lchild);// 左子女入队列
       if(p->rchild)                 // 若右子女非空
         if(p->rchild->data==x)      //右子女结点值为 x,应删除当前结点的右
子树
          {DeleteXTree(p->rchild);   p->rchild=null;} //父结点的右子女置空
         else Enqueue (Q,p->rchild);// 右子女入队列
      }//while
    }//if(bt)  }//search
```

9. int BTLC(BiTree T,int *c)//对二叉树 T 的结点计数

```
{if(T)
   {*c++;
    BTLC(T->lchild,&c); //统计左子树结点
    BTLC(T->rchild,&c); //统计右子树结点
    } }//结束 Count,调用时*c=0
```

10.

（1）找结点的中序前驱结点

```
BiTNode  *InPre (BiTNode  *p)
/*在中序线索二叉树中查找 p 的中序前驱结点，并用 pre 指针返回结果*/
{ if (p->Ltag= =1)  pre = p->LChild;   /*直接利用线索*/
 else
   {/*在 p 的左子树中查找“最右下端”结点*/
     for ( q=p->LChild; q->Rtag= =0; q=q->RChild);
      pre = q;
    }
 return (pre);
}
```

（2）找结点的中序后继结点

```
BiTNode  *InSucc (BiTNode  *p)
/*在中序线索二叉树中查找 p 的中序后继结点，并用 succ 指针返回结果*/
{ if (p->Rtag= =1)  succ = p->RChild;   /*直接利用线索*/
 else
   {/*在 p 的右子树中查找“最左下端”结点*/
     for ( q=p->RChild; q->Ltag= =0; q=q->LChild);
      succ= q;
    }
 return (succ);
}
```

(3) 找结点的先序后继结点

```
BiTNode  *PreSucc (BiTNode  *p)
/*在先序线索二叉树中查找 p 的先序后继结点，并用 succ 指针返回结果*/
{ if (p->Ltag= =0)  succ = p->LChild;
 else   succ= p->RChild;
 return (succ);
}
```

(4) 找结点的后序前驱结点

```
BiTNode  *SuccPre (BiTNode  *p)
/*在后序线索二叉树中查找 p 的后序前驱结点，并用 pre 指针返回结果*/
{ if (p->Ltag= =1)  pre = p->LChild;
 else   pre= p->RChild;
 return (pre);
}
```

# 第七章　图

## 一、单项选择题

1.B
2.B
3．B
4．A
5．B
6．B
7．B C
8．B
9．C
10．B
11．B
12．A
13．D
14．A
15．B

## 二、填空题

1．45
2．略
3．n
4．3
5．2（N-1）
6．第 I 列非零元素个数
7．n　2e
8．深度优先
9．普里姆（prim）算法和克鲁斯卡尔（Kruskal）算法
10．不存在环
11．递增　负值
12．50，经过中间顶点④
13.(1)活动　（2）活动间的优先关系　（3）事件　（4）活动　边上的权代表活动持续时间
14．关键路径
15．（1）某项活动以自己为先决条件　（2）荒谬　（3）死循环
16．1）V1 V4 V3 V6 V2 V5(尽管图以邻接表为存储结构，但因没规定邻接点的排列，所以结果是不唯一的。本答案是按邻接点升序排列给出的。)
（2）① top==-1　　② top=graph[j].count　　③ graph[k].count==0

## 三、应用题

1.【解答】

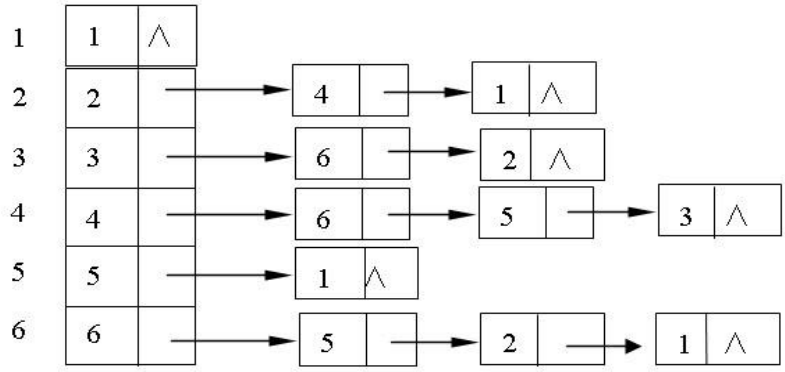
| （1）顶点 | 入度 | 出度 |
|---|---|---|
| 1 | 3 | 0 |
| 2 | 2 | 2 |

| | | |
|---|---|---|
| 3 | 1 | 2 |
| 4 | 1 | 3 |
| 5 | 2 | 1 |
| 6 | 2 | 3 |

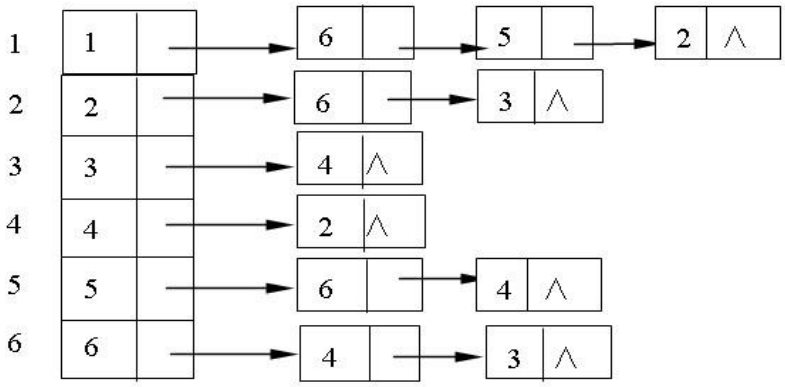
（2） 邻接矩阵

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

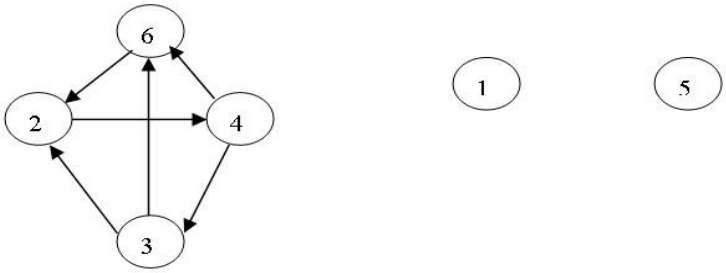
（3）邻接表




（4）逆邻接表



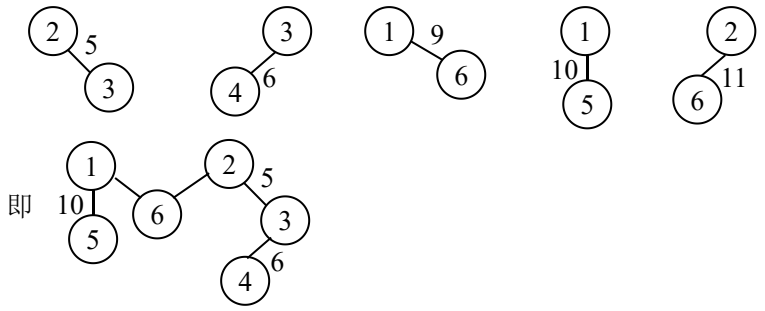

（5）强连通分量



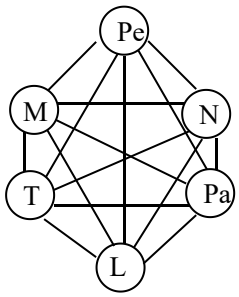

2．略

3．为节省篇幅，这里仅用 Kruskal 算法，构造最小生成树过程如下：（下图也可选(2,4)代

替(3,4),(5,6)代替(1,5))




4. (1)




(2)略

(3)最小生成树 6 个顶点 5 条边：

V(G)={Pe,N,Pa,L,T,M}

E(G)={(L,Pa,3),(Pe,T,21),(M,N,32),(L,N,55),(L,Pe,81)}

5.（1）AOE 网图略

（2）各事件发生的最早和最晚时间如下表

| 事　　件 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 最早发生时间 | 0 | 15 | 10 | 65 | 50 | 80 | 200 | 380 | 395 | 425 | 445 | 420 |
| 最晚发生时间 | 0 | 15 | 57 | 65 | 380 | 80 | 335 | 380 | 395 | 425 | 445 | 425 |

（3）关键路径为顶点序列：1->2->4->6->8->9->10->11；

事件序列：A->C->E->G->H->L->M，完成工程所需的最短时间为 445。

## 四、算法设计题

1.
```
void CreatAdjList(AdjList g)
 //建立有向图的邻接表存储结构
{int n;
 scanf("%d",&n);
 for (i=1;i<=n;j++)
   {scanf(&g[i].vertex); g[i].firstarc=null;}//输入顶点信息
   scanf(&v1,.&v2);
   while(v1 && v2)//题目要求两顶点之一为 0 表示结束
    {i=GraphLocateVertex(g2,v1);
     p=(ArcNode*)malloc(sizeof(ArcNode));
     p->adjvex=j;  p->next=g[i].firstarc;  g[i].firstarc=p;
    scanf(&v1,&v2);
```

```
        } }
```

2. void AdjMatrixToAdjList( AdjMatrix gm, AdjList gl )

```
        //将图的邻接矩阵表示法转换为邻接表表示法。
    {for (i=1;i<=n;i++)   //邻接表表头向量初始化。
      {scanf(&gl[i].vertex); gl[i].firstarc=null;}
     for (i=1;i<=n;i++)
       for (j=1;j<=n;j++)
          if (gm[i][j]==1)
             {p=(ArcNode *)malloc(sizeof(ArcNode)) ;//申请结点空间。
               p->adjvex=j;//顶点 I 的邻接点是 j
               p->next=gl[i].firstarc; gl[i].firstarc=p; //链入顶点 i 的邻接点链表中
               }
       }//end
```

[算法讨论] 算法中邻接表中顶点在向量表中的下标与其在邻接矩阵中的行号相同。

3. void DeletEdge(AdjList g,int i,j)

```
     //在用邻接表方式存储的无向图 g 中，删除边(i,j)
    {p=g[i].firstarc;  pre=null; //删顶点 i 的边结点(i,j),pre 是前驱指针
      while (p)
       if (p->adjvex==j)
         {if(pre==null)g[i].firstarc=p->next;else pre->next=p->next;free(p);}//释放结点空间。
       else {pre=p; p=p->next;}   //沿链表继续查找
      p=g[j].firstarc;  pre=null; //删顶点 j 的边结点(j,i)
      while (p)
       if (p->adjvex==i)
         {if(pre==null)g[j].firstarc=p->next;else pre->next=p->next;free(p);}//释放结点空间。
       else {pre=p; p=p->next;}   //沿链表继续查找
     }// DeletEdge
```

[算法讨论] 算法中假定给的 i，j 均存在，否则应检查其合法性。若未给顶点编号，而给出顶点信息，则先用顶点定位函数求出其在邻接表顶点向量中的下标 i 和 j。

4. [题目分析]有向图判断回路要比无向图复杂。利用深度优先遍历，将顶点分成三类：未访问；已访问但其邻接点未访问完;已访问且其邻接点已访问完。下面用 0，1，2 表示这三种状态。前面已提到，若 dfs（v）结束前出现顶点 u 到 v 的回边，则图中必有包含顶点 v 和 u 的回路。对应程序中 v 的状态为 1，而 u 是正访问的顶点，若我们找出 u 的下一邻接点的状态为 1，就可以输出回路了。

```
  void  Print(int v,int start )  //输出从顶点 start 开始的回路。
    {for(i=1;i<=n;i++)
      if(g[v][i]!=0 && visited[i]==1 )  //若存在边（v,i)，且顶点 i 的状态为 1。
        {printf(“%d”,v);       if(i==start)       printf(“\n”);       else
```

```
Print(i,start);break;}//if
    }//Print
  void dfs(int v)
   {visited[v]=1;
    for(j=1;j<=n;j++ )
       if (g[v][j]!=0) //存在边(v,j)
         if (visited[j]!=1) {if (!visited[j]) dfs(j); }//if
         else {cycle=1; Print(j,j);}
    visited[v]=2;
    }//dfs
  void find_cycle() //判断是否有回路，有则输出邻接矩阵。visited 数组为全局变量。
     {for (i=1;i<=n;i++) visited[i]=0;
      for (i=1;i<=n;i++ ) if (!visited[i])  dfs(i);
     }//find_cycle
```

5.［题目分析］该题可用求每对顶点间最短路径的 FLOYD 算法求解。求出每一顶点（村庄）到其它顶点（村庄）的最短路径。在每个顶点到其它顶点的最短路径中，选出最长的一条。因为有 n 个顶点，所以有 n 条，在这 n 条最长路径中找出最短一条，它的出发点（村庄）就是医院应建立的村庄。

```
   void  Hospital(AdjMatrix w,int n)
    //在以邻接带权矩阵表示的 n 个村庄中，求医院建在何处，使离医院最远的村庄到医院的路径最短。
    {for (k=1;k<=n;k++)   //求任意两顶点间的最短路径
       for (i=1;i<=n;i++)
         for (j=1;j<=n;j++)
           if (w[i][k]+w[k][j]<w[i][j])  w[i][j]=w[i][k]+w[k][j];
     m=MAXINT;             //设定 m 为机器内最大整数。
     for (i=1;i<=n;i++)    //求最长路径中最短的一条。
       {s=0;
        for (j=1;j<=n;j++) //求从某村庄 i（1<=i<=n）到其它村庄的最长路径。
          if (w[i][j]>s) s=w[i][j];
        if (s<=m) {m=s; k=i;}//在最长路径中，取最短的一条。m 记最长路径，k 记出发顶点的下标。
        Printf(“医院应建在%d 村庄，到医院距离为%d\n”,i,m);
        }//for
    }//算法结束
```

对以上实例模拟的过程略。在 $A^{(6)}$矩阵中（见下图），各行中最大数依次是 9，9，6，7，9，9。这几个最大数中最小者为 6，故医院应建在第三个村庄中,离医院最远的村庄到医院的距离是 6。

6.［题目分析］对有向图进行深度优先遍历可以判定图中是否有回路。若从有向图某个顶点 v 出发遍历，在 dfs(v)结束之前，出现从顶点 u 到顶点 v 的回边，图中必存在环。这里设定 visited 访问数组和 finished 数组为全局变量，若 finished[i]=1,表示顶点 i 的邻接点已搜索完毕。由于 dfs 产生的是逆拓扑排序，故设一类型是指向邻接表的边结点的全局指针变

量 final，在 dfs 函数退出时，把顶点 v 插入到 final 所指的链表中，链表中的结点就是一个正常的拓扑序列。邻接表的定义与本书相同，这里只写出拓扑排序算法。

```
int visited[]=0; finished[]=0; flag=1;   //flag 测试拓扑排序是否成功
ArcNode  *final=null;    //final 是指向顶点链表的指针，初始化为 0
void  dfs(AdjList g,vertype v)
   //以顶点 v 开始深度优先遍历有向图 g，顶点信息就是顶点编号.
  {ArcNode *t;  //指向边结点的临时变量
   printf("%d",v); visited[v]=1; p=g[v].firstarc;
   while(p!=null)
     {j=p->adjvex;
      if (visited[j]==1 && finished[j]==0) flag=0 //dfs 结束前出现回边
        else if(visited[j]==0) {dfs(g,j); finished[j]=1;} //if
      p=p->next;
     }//while
   t=(ArcNode *)malloc(sizeof(ArcNode));//申请边结点
   t->adjvex=v; t->next=final; final=t;    //将该顶点插入链表
  } //dfs 结束
int dfs-Topsort(Adjlist g)
  //对以邻接表为存储结构的有向图进行拓扑排序，拓扑排序成功返回 1，否则返回 0
  {i=1;
   while (flag && i <=n)
     if (visited[i]==0) {dfs(g,i);  finished[i]=1; }//if
   return(flag);
    }// dfs-Topsort
```

# 第九章　查找

## 一、单项选择题

1．B
2．C
3．D
4．C
5．B
6．B
7．C C
8．A
9．C
10．B
11．A
12．D
13．B
14．D
15．C

## 二、填空题

1．n　n+1
2．4
3．　二叉排序_
4．(1)哈希函数(2)解决冲突的方法 (3)选择好的哈希函数 (4)处理冲突的方法 (5)均匀(6)简单
5．AVL 树（高度平衡树，高度平衡的二叉排序树）
　　或为空二叉树，或二叉树中任意结点左子树高度与右子树高度差的绝对值小于等于 1。
6．小于等于表长的最大素数或不包含小于 20 的质因子的合数
7．k(k+1)/2
8．直接定址法
9．插入　删除
10．二叉排序树
11．　存取元素时发生冲突的可能性就越大　存取元素时发生冲突的可能性就越小
12．(1)rear=mid-1 (2)head=mid+1　(3)head>rear

## 三、应用题

1．哈希方法的平均查找路长主要取决于负载因子（表中实有元素数与表长之比），它反映了哈希表的装满程度，该值一般取 0.65~0.9。

散列表存储中解决碰撞的基本方法：

①　开放定址法　形成地址序列的公式是：$H_i$=（H（key）+$d_i$）% m，其中 m 是表长，$d_i$是增量。根据 $d_i$取法不同，又分为三种：

a．$d_i$ =1，2，…，m-1 称为线性探测再散列，其特点是逐个探测表空间，只要散列表中有空闲空间，就可解决碰撞，缺点是容易造成“聚集”，即不是同义词的关键字争夺同一散列地址。

b．$d_i$ =$1^2$，$-1^2$，$2^2$，$-2^2$，… ，$\pm k^2$（k≤m/2） 称为二次探测再散列，它减少了聚集，但不容易探测到全部表空间，只有当表长为形如 4j+3（j 为整数）的素数时才有可能。

c．$d_i$ =伪随机数序列，称为随机探测再散列。

②　再散列法　$H_i$=$RH_i$（key） i=1，2，…，k，是不同的散列函数，即在同义词产生碰撞时，用另一散列函数计算散列地址，直到解决碰撞。该方法不易产生“聚集”，但增加了计算时间。

③　链地址法　将关键字为同义词的记录存储在同一链表中，散列表地址区间用H[0..m-1]表示，分量初始值为空指针。凡散列地址为i（0≤i≤m-1）的记录均插在以H[i]为头指针的链表中。这种解决方法中数据元素个数不受表长限制，插入和删除操作方便，但增加了指针的空间开销。这种散列表常称为开散列表，而①中的散列表称闭散列表，含义是元素个数受表长限制。

④　建立公共溢出区　设H[0..m-1]为基本表，凡关键字为同义词的记录，都填入溢出区
O[0..m-1]。

2.

| 散列地址 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| 关键字 | 14 | 01 | 9 | 23 | 84 | 27 | 55 | 20 | | |
| 比较次数 | 1 | 1 | 1 | 2 | 3 | 4 | 1 | 2 | | |

平均查找长度：$ASL_{succ}$=（1+1+1+2+3+4+1+2）/8=15/8

以关键字27为例：H（27）=27%7=6（冲突）　$H_1$=（6+1）%10=7（冲突）

$H_2$=（6+$2^2$）%10=0（冲突）　$H_3$=（6+$3^3$）%10=5　所以比较了4次。

3.　表略

$ASL_{succ}$ =15/10

4. 略

5. 在二叉排序树上查找关键字K，走了一条从根结点至多到叶子的路径，时间复杂度是O(logn)，而在中序遍历输出的序列中查找关键字K，时间复杂度是O（n）。按序输入建立的二叉排序树，蜕变为单枝树，其平均查找长度是（n+1）/2，时间复杂度也是O（n）。

6. 略
7. 略

**四、算法设计题**

1. int　Search（rectype r[ ]，int　n，keytype k）

```
//在n个关键字从小到大排列的顺序表中，查找关键字为k的结点。
{r[n+1].key=MAXINT; //在高端设置监视哨
 i=1;
 while（r[i].key<k）i++;
 if (r[n+1].key==k) return （i%（n+1））;
 else return (0)
}//算法search结束
```

查找过程的判定树是单枝树，限于篇幅不再画出。本题中虽然表按关键字有序，但进行顺序查找，查找成功的平均查找长度亦为（n+1）/2。

2. 解答:. 分析：采用递归方法，从根结点开始查找结点p，若查询结点是所要找的结点，返回其深度h0。否则，到左、右子树上去找，查找深度加1。

```
int find1 ( birtreptr T , p , int h0 )
/*在二叉树排序树T中查找结点p的层次，若不在时返回空值。h0为根结点T的层次*/
{ if ( p == NULL ) return ( 0 ) ;                    /* 没找到，返回0 */
```

```
    if( p ==T)      return( h0 ) ;                /* 找到 */
    else if (p -> data < T -> data ) return( find1( T ->lchild,p,h0) +1) ; /*
到左子树去找 */
        else return ( find1 ( T -> rchild,p,h0) + 1);              /* 到右子
树去找 */
  }
  int find2( birtrptr T, p )  { return ( find1 ( T, p,1 ) ) ; }
```

3. **void** Delete（BSTree t，p）

```
    // 在二叉排序树 t 中，删除 f 所指结点的右孩子（由 p 所指向）的算法
    {if（p->lchild==null）{f->rchild=p->rchild; free（p）; } //p 无左子女
     else   //用 p 左子树中的最大值代替 p 结点的值
       {q=p->lchild; s=q;
        while（q->rchild）{s=q; q=q->rchild ; }//查 p 左子树中序序列最右结点
        if（s==p->lchild） //p 左子树的根结点无右子女
          {p->data=s->data; p->lchild=s->lchild; free（s）; }
        else{p->data=q->data; s->rchild=q->lchild; free（q）; }
        }
     }//Delete
```

# 第 10 章　排序

## 一、单项选择题

1.（1） DC　（2）ADF　（3）B　（4）ACF　BDE
2. D
3. A
4. C
5. C
6. C
7. D
8. A
9. C
10. D
11. C
12. C

## 二、填空题

1.稳定、不稳定
2.内部、外部
3. 免去查找过程中每一步都要检测整个表是否查找完毕，提高了查找效率。
4. n(n-1)/2
5. 题中为操作方便，先增加头结点（最后删除），p 指向无序区的前一记录，r 指向最小值结点的前驱，一趟排序结束，无序区第一个记录与 r 所指结点的后继交换指针。
(1)q->link!=NULL (2)r!=p　(3)p->link (4)p->link=s　(5)p=p->link
6 . .(1)i<n-i+1　(2)j<=n-i+1　(3)r[j].key<r[min].key　(4)min!=i　(5)max==i (6)r[max]<-->r[n-i+1]
7. (1)1　(2)a[i]=t　(3)(i=2;i<=n;i+=2)　(4)1　(5)flag

## 三、应用题

1. 解答：①直接插入排序

| 序号 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 关键字 | 83 | 40 | 63 | 13 | 84 | 35 | 96 | 57 | 39 | 79 | 61 | 15 |
| i = 2 | 40 | 83 | [63 | 13 | 84 | 35 | 96 | 57 | 39 | 79 | 61 | 15] |
| i = 3 | 40 | 63 | 83 | [13 | 84 | 35 | 96 | 57 | 39 | 79 | 61 | 15] |
| i = 4 | 13 | 40 | 63 | 83 | [84 | 35 | 96 | 57 | 39 | 79 | 61 | 15] |
| i = 5 | 13 | 40 | 63 | 83 | 84 | [35 | 96 | 57 | 39 | 79 | 61 | 15] |
| i = 6 | 13 | 35 | 40 | 63 | 83 | 84 | [96 | 57 | 39 | 79 | 61 | 15] |
| i = 7 | 13 | 35 | 40 | 63 | 83 | 84 | 96 | [57 | 39 | 79 | 61 | 15] |
| i = 8 | 13 | 35 | 40 | 57 | 63 | 83 | 84 | 96 | [39 | 79 | 61 | 15] |
| i = 9 | 13 | 35 | 39 | 40 | 57 | 63 | 83 | 84 | 96 | [79 | 61 | 15] |
| i = 10 | 13 | 35 | 39 | 40 | 57 | 63 | 79 | 83 | 84 | 96 | [61 | 15] |
| i = 11 | 13 | 35 | 39 | 40 | 57 | 61 | 63 | 79 | 83 | 84 | 96 | [15] |
| i = 12 | 13 | 15 | 35 | 39 | 40 | 57 | 61 | 63 | 79 | 83 | 84 | 96 |

②直接选择排序

| 序号 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 关键字 | 83 | 40 | 63 | 13 | 84 | 35 | 96 | 57 | 39 | 79 | 61 | 15 |
| i = 1 | 13 | [40 | 63 | 83 | 84 | 35 | 96 | 57 | 39 | 79 | 61 | 15] |
| i = 2 | 13 | 15 | [63 | 83 | 84 | 35 | 96 | 57 | 39 | 79 | 61 | 40] |
| i = 3 | 13 | 15 | 35 | [83 | 84 | 63 | 96 | 57 | 39 | 79 | 61 | 40] |
| i = 4 | 13 | 15 | 35 | 39 | [84 | 63 | 96 | 57 | 83 | 79 | 61 | 40] |
| i = 5 | 13 | 15 | 35 | 39 | 40 | [63 | 96 | 57 | 83 | 79 | 61 | 84] |
| i = 6 | 13 | 15 | 35 | 39 | 40 | 57 | [96 | 63 | 83 | 79 | 61 | 84] |
| i = 7 | 13 | 15 | 35 | 39 | 40 | 57 | 61 | [63 | 83 | 79 | 96 | 84] |
| i = 8 | 13 | 15 | 35 | 39 | 40 | 57 | 61 | 63 | [83 | 79 | 96 | 84] |
| i = 9 | 13 | 15 | 35 | 39 | 40 | 57 | 61 | 63 | 79 | [83 | 96 | 84] |
| i = 10 | 13 | 15 | 35 | 39 | 40 | 57 | 61 | 63 | 79 | 83 | [96 | 84] |
| i = 11 | 13 | 15 | 35 | 39 | 40 | 57 | 61 | 63 | 79 | 83 | 84 | [96] |

③快速排序

| 关键字 | 83 | 40 | 63 | 13 | 84 | 35 | 96 | 57 | 39 | 79 | 61 | 15 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 第一趟排序后 | [15 | 40 | 63 | 13 | 61 | 35 | 79 | 57 | 39] | 83 | [96 | 84] |
| 第二趟排序后 | [13] | 15 | [63 | 40 | 61 | 35 | 79 | 57 | 39] | 83 | 84 | [96] |
| 第三趟排序后 | 13 | 15 | [39 | 40 | 61 | 35 | 57] | 63 | [79] | 83 | 84 | 96 |
| 第四趟排序后 | 13 | 15 | [35] | 39 | [61 | 40 | 57] | 63 | 79 | 83 | 84 | 96 |
| 第五趟排序后 | 13 | 15 | 35 | 39 | [57 | 40] | 61 | 63 | 79 | 83 | 84 | 96 |
| 第六趟排序后 | 13 | 15 | 35 | 39 | 40 | [57] | 61 | 63 | 79 | 83 | 84 | 96 |
| 第七趟排序后 | 13 | 15 | 35 | 39 | 40 | 57 | 61 | 63 | 79 | 83 | 84 | 96 |

④堆排序

关键字： 83 40 63 13 84 35 96 57 39 79 61 15

排序成功的序列：96 84 83 79 63 61 57 40 39 35 15 13

排序过程如图简答题 8－1.1、8－1.2、8－1.3 所示。

⑤归并排序

| 关键字 | 83 | 40 | 63 | 13 | 84 | 35 | 96 | 57 | 39 | 79 | 61 | 15 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 第一趟排序后 | [40 | 83] | [13 | 63] | [35 | 84] | [57 | 96] | [39 | 79] | [15 | 61] |
| 第二趟排序后 | [13 | 40 | 63 | 83] | [35 | 57 | 84 | 96] | [15 | 39 | 61 | 79] |
| 第三趟排序后 | [13 | 35 | 40 | 57 | 63 | 83 | 84 | 96] | [15 | 39 | 61 | 79] |
| 第四趟排序后 | 13 | 15 | 35 | 39 | 40 | 57 | 61 | 63 | 79 | 83 | 84 | 96 |

2．(1)是大堆； (2)是大堆；（4）是小堆；

(3)不是堆，调成大堆 100，98，66，85，80，60，40，77，82，10，20

3．(1)①child=child+1; ②child/2

(2)不能，调为大堆：92,86,56,70,33,33,48,65,12,24

**四、算法设计题：**

1．解答：分析：每趟从单链表头部开始，顺序查找当前链值最小的结点。找到后，插入到当前的有序表区的最后。

```
Void selesort ( lklist L )              /* 设链表 L 带头结点 */
```

```
        {  q=L;                          /* 指向第一数据前趋 */
          while ( q-> next !=NULL )
             {  pl = q -> next ;
                minp =pl;              /* minp 指向当前已知的最小数 */
                while ( pl -> next != NULL )
                   { if ( pl -> next -> data < minp -> data )
                     minp = pl -> next ; /* 找到了更小数 */
                     pl = pl -> next ;   /* 继续往下找 */
                    }
                 if ( minp != q -> next)  /* 将最小数交换到第一个位置上 */
                    { r1 = minp -> next ;
                      minp -> next = r1 -> next ;        /* 删除最小数 */
                      r2 = q -> next ;
                      q -> next = r2 -> next ; /* 删除当前表中第一个数 */
                      r1 -> next = q -> next ;
                      q -> next = r1 ;        /* 将最小数插入到第一个位置上 */
                      r2 -> next = minp -> next ;
                      minp -> next = r2 ;    /* 将原第一个数放到最小数原位置上
  */
                     }
                  q = q > next ;              /* 选择下一个最小数 */
               }
    }
```

2. void BubbleSort2(int a[],int n) //相邻两趟向相反方向起泡的冒泡排序算法

```
  { change=1;low=0;high=n-1; //冒泡的上下界
    while(low<high && change)
    { change=0;              //设不发生交换
      for(i=low;i<high;i++)  //从上向下起泡
        if(a[i]>a[i+1]){a[i]<-->a[i+1];change=1;} //有交换，修改标志 change
      high--; //修改上界
      for(i=high;i>low;i--) //从下向上起泡
        if(a[i]<a[i-1]){a[i]<-->a[i-1];change=1;}
      low++; //修改下界
     }//while
  }//BubbleSort2
```

[算法讨论]题目中“向上移”理解为向序列的右端，而“向下移”按向序列的左端来处理。

3. typedef struct

```
     { int num; float score; }RecType;
  void SelectSort(RecType R[51]，int n)
  { for(i=1; i<n; i++)
      { //选择第 i 大的记录，并交换到位
```

```
        k=i;        //假定第 i 个元素的关键字最大
        for(j=i+1;j<=n;j++)      //找最大元素的下标
          if(R[j].score>R[k].score)  k=j;
        if(i!=k)  R[i] <-->R[k];  //与第 i 个记录交换
      }//for
    for(i=1; i<=n; i++) //输出成绩
      { printf("%d,%f",R[i].num,R[i].score); if(i%10==0) printf("\n");}
  }//SelectSort
```

4. 解答：分析：此问题分为两个算法，第一个算法 shift 假设 a[ 1] , a[ 2 ],…,a[ k] 为堆，增加一个无素 a[ k + 1 ],把数组 a[ 1 ],a[ 2 ], …,a[ k + 1 ]调整为堆。第二个算法 heep 从 1 开始调用算法 sift 将整个数组调整为堆。

```
Void sift (datatype A[ n ] , int  K)              /* n > = k + 1 */
    {  x = A[ K+1] ;
       i = K +1 ;
       while ( ( i/2 > 0 )&&( A[i/2]>x) ) { A[i]= A[i./2]; i = i/2;} /*从下往上插入位置 */
       A[i] = x ;
    }
Void heap ( datatype A[ n ] ) ;  /* 从 1 开始调用算法 sift ，将整个数组调整为堆 */
  { for ( k = 1 ; k <= n-1; k++ ) sift ( A,k ) ; }
```

# 第十一章　文件

**一、单项选择题**

1．A
2．D
3．A
4．B
5．C

**二、填空题**

1. 操作系统文件　　数据库
2. 单关键字文件　　多关键字文件
3. 记录　数据项
4. 第 I-1
5. 随机
6. 构造散列函数　解决冲突的方法
7. 分配和释放存储空间　　重组　　对插入的记录

**三、应用题**

1. 在索引文件中，若（数据区）主文件中关键字有序，则文件称为索引顺序文件。索引顺序文件因主文件有序，一般用稀疏索引，占用空间较少。常用索引顺序文件有 ISAM 和 VSAM。ISAM 采用静态索引结构，而 VSAM 采用 B+树的动态索引结构。

2.（1）顺序文件只能顺序查找，优点是批量检索速度快，不适于单个记录的检索。顺序文件不能象顺序表那样插入、删除和修改，因文件中的记录不能象向量空间中的元素那样“移动”，只能通过复制整个文件实现上述操作。
（2）索引非顺序文件适合随机存取，不适合顺序存取，因主关键字未排序，若顺序存取会引起磁头频繁移动。索引顺序文件是最常用的文件组织，因主文件有序，既可顺序存取也可随机存取。索引非顺序文件是稠密索引，可以“预查找”，索引顺序文件是稀疏索引，不能“预查找”，但由于索引占空间较少，管理要求低，提高了索引的查找速度。
（3）散列文件也称直接存取文件，根据关键字的散列函数值和处理冲突的方法，将记录散列到外存上。这种文件组织只适用于像磁盘那样的直接存取设备，其优点是文件随机存放，记录不必排序，插入、删除方便，存取速度快，无需索引区，节省存储空间。缺点是散列文件不能顺序存取，且只限于简单查询。经多次插入、删除后，文件结构不合理，需重组文件，这很费时。

3．[问题分析]在职务项中增加一个指针项，指向其领导者。因题目中未提出具体的隶属关系，如哪个系的系主任，哪个系哪个室的室主任，哪个室的教员等。这里假设每个室主任隶属于他前边离他最近的那个系主任，每个教员隶属于他前边离他最近的那个室主任，见下面多重表文件。在职称项中增加一个指针项，指向同一职称的下一个职工，增加一个次关键字索引表：

| 关键字 | 头指针 | 长度 |
|---|---|---|
| 讲师 | 001 | 2 |

| | | |
|---|---|---|
| 副教授 | 004 | 2 |
| 教授 | 002 | 6 |

“职称”索引表

| 记录号 | 职工号 | 职工姓名 | 职务 | | 职称 | |
|---|---|---|---|---|---|---|
| 01 | 001 | 张军 | 教员 | 04 | 讲师 | 08 |
| 02 | 002 | 沈灵 | 系主任 | 03 | 教授 | 03 |
| 03 | 003 | 叶明 | 校长 | ∧ | 教授 | 05 |
| 04 | 004 | 张莲 | 室主任 | 02 | 副教授 | 10 |
| 05 | 005 | 叶宏 | 系主任 | 03 | 教授 | 06 |
| 06 | 006 | 周芳 | 教员 | 04 | 教授 | 07 |
| 07 | 007 | 刘光 | 系主任 | 03 | 教授 | 09 |
| 08 | 008 | 黄兵 | 教员 | 04 | 讲师 | ∧ |
| 09 | 009 | 李民 | 室主任 | 07 | 教授 | ∧ |
| 10 | 010 | 赵松 | 教员 | 09 | 副教授 | ∧ |
| | … | … | … | | … | |

多重表文件